

Scaling Spike-driven Transformer with Efficient Spike Firing Approximation Training

Man Yao*, Xuerui Qiu*, Tianxiang Hu, Jiakui Hu, Yuhong Chou, Keyu Tian, Jianxing Liao, Luziwei Leng, Bo Xu, and Guoqi Li†

Abstract—The ambition of brain-inspired Spiking Neural Networks (SNNs) is to become a low-power alternative to traditional Artificial Neural Networks (ANNs). This work addresses two major challenges in realizing this vision: the performance gap between SNNs and ANNs, and the high training costs of SNNs. We identify intrinsic flaws in spiking neurons caused by binary firing mechanisms and propose a Spike Firing Approximation (SFA) method using integer training and spike-driven inference. This optimizes the spike firing pattern of spiking neurons, enhancing efficient training, reducing power consumption, improving performance, enabling easier scaling, and better utilizing neuromorphic chips. We also develop an efficient spike-driven Transformer architecture and a spike-masked autoencoder to prevent performance degradation during SNN scaling. On ImageNet-1k, we achieve state-of-the-art top-1 accuracy of 78.5%, 79.8%, 84.0%, and 86.2% with models containing 10M, 19M, 83M, and 173M parameters, respectively. For instance, the 10M model outperforms the best existing SNN by 7.2% on ImageNet, with training time acceleration and inference energy efficiency improved by $4.5\times$ and $3.9\times$, respectively. We validate the effectiveness and efficiency of the proposed method across various tasks, including object detection, semantic segmentation, and neuromorphic vision tasks. This work enables SNNs to match ANN performance while maintaining the low-power advantage, marking a significant step towards SNNs as a general visual backbone.

Index Terms—Spiking neural network, Spike-driven, Spiking Transformer, Neuromorphic computing, Efficient architecture and training

I. INTRODUCTION

SPIKING neural networks (SNNs) have garnered significant attention due to their brain-inspired spatio-temporal dynamics and spike-driven efficient computing paradigm [1], [2]. Spiking neurons communicate through binary spikes, following an “integrate-and-fire” dynamic model [3]. Spatial and temporal information are integrated into membrane potentials by spiking neurons. When the membrane potential surpasses a certain threshold, the neuron emits a spike, and the membrane potential is reset according to specific rules. Neuromorphic chips [4]–[6] use spiking neurons and synapses as fundamental

computing units to implement SNNs on hardware circuits. The spike-driven nature of neuromorphic computing enables SNNs to trigger sparse additions only when a signal is present, offering significant power efficiency advantages over traditional Artificial Neural Networks (ANNs). This low-power advantage is particularly evident in small-scale edge computing scenarios [7], [8]. For instance, the recently introduced asynchronous sensing-computing neuromorphic System on Chip (SoC) Speck [9] has a resting power of just 0.42mW, and its operational power consumption in typical neuromorphic vision applications ranges from 0.7 to 15 mW.

In the era of deep learning, model scale is crucial to the success of modern machine intelligence. SNNs are also scaling up. There are two main training methods for large-scale SNNs: ANN2SNN and direct training. ANN2SNN approximates ANN activations using spike firing rates, achieving similar accuracy but requires numerous inference timesteps and abandoning spatio-temporal dynamics [10]–[14]. Direct training converts the nonlinear differential equations of spiking neurons into a discrete iterable form, and utilizing Spatio-Temporal BackPropagation (STBP) and surrogate gradients for training [15], [16]. This method offers flexibility in architecture design and leverages spatio-temporal dynamics of spiking neurons [17]–[24], but multi-timestep simulations demand substantial computing resources. For example, direct training a spiking ResNet-19 with 10 timesteps requires approximately $20\times$ more memory than a traditional ResNet-19 [25].

On the path to scaling SNNs, current training methods face a paradox. Extended timesteps reduce errors from quantizing membrane potential into spikes, ensuring better task performance. However, multi-timestep simulations demand substantial GPU memory and are considerably more challenging to train, thereby impeding the scalability and performance improvement of SNNs.

Existing views simply think that transforming continuous potential into spikes results in quantization errors [26]. We extend this understanding by showing that binary firing is a fundamental mechanism flaw and is the source of paradox. The binary nature of spike firing impairs the spiking neuron’s ability to assess the significance of incoming signals (spatial representation), and constrains its forgetting capabilities (temporal dynamics). Current methods, including ANN2SNN and direct training, attempt to address the spatial representation via rate coding [27] but remain inefficient training and unable to overcome temporal dynamics limitations.

We propose a simple Spike Firing Approximation (SFA) method to solve these flaws. SFA uses integers as activations

M. Yao and X. Qiu and T. Hu, B.Xu and G.Li are with Institute of Automation, Chinese Academy of Sciences, Beijing, China.

X. Qiu is also with School of Future Technology, University of Chinese Academy of Sciences, Beijing, China.

J. Hu and K.Tian are with the Institute for Artificial Intelligence, Peking University, Beijing, China.

Y. Chou is with Department of Computing, The Hong Kong Polytechnic University, HongKong, China.

J. Liao and L. Leng are with Huawei ACS Lab, Shenzhen, Guangdong, China.

* Equal contribution (E-mail: man.yao@ia.ac.cn; qiuxuerui2024@ia.ac.cn).

† Corresponding author: Guoqi Li (E-mail: guoqi.li@ia.ac.cn).

during training, which are then converted to spike trains during inference. This results in a fundamentally different spike firing pattern compared to ANN2SNN and direct training. Unlike the synchronous and random firing of the latter two, the SFA firing pattern is asynchronous and continuous. We demonstrate that this firing pattern significantly affects training, inference power, performance, network scaling, and neuromorphic chip implementation simultaneously. In all these aspects, the SFA approach outperforms ANN2SNN and direct training.

We then focus on further enhancing the performance upper bound of SNNs through architecture design and model scale expansion. Meta-SpikeFormer [28] investigates the meta-design of SNN architecture based on spike-driven Transformer [29], which for the first time integrates the spike-driven paradigm into Transformer. We upgrade the Meta-SpikeFormer, including improving the convolution and spike-driven self-attention operators, etc. The proposed Efficient Spiking Transformer (E-SpikeFormer) can simultaneously improve performance and energy efficiency. To solve the degradation problem of E-SpikeFormer when scaling up, we design a spike sparse convolution and combine it with a masked image modeling [30], [31] pre-training strategy.

The proposed methods are validated on both static image classification (ImageNet-1k [32]), object detection (COCO [33]), semantic segmentation (ADE20K [34]), and neuromorphic action recognition (HAR-DVS [35]) tasks. We scale up SNNs and achieve state-of-the-art results on all tested datasets. Our main contributions are:

- **Analysis of Spiking Neuron Mechanisms:** We identify binary spike firing as a mechanistic defect impacting both the spatial representation and the temporal dynamics of spiking neurons.
- **Spike Firing Approximation (SFA):** The proposed SFA method leverages the equivalence between integer-based training and spike train inference. We demonstrate that SFA simultaneously optimizes training, power efficiency, performance, and scaling of SNNs by adjusting the spike firing patterns of spiking neurons. We also discuss the implementations of SFA on neuromorphic chips.
- **Efficient architecture design:** We introduce the E-SpikeFormer, which achieves higher accuracy with lower energy cost based on the prior state-of-the-art SNN by designing efficient spike-driven convolution and self-attention operators.
- **Scaling SNNs.** We integrate spike sparse convolution and masked image modeling pre-training to address the challenge of performance degradation when scaling SNNs.
- **Enhancing the performance upper bound of SNNs:** We show that SNNs can achieve performance comparable to ANNs across various vision tasks with lower energy consumption, highlighting their potential as a general low-power visual backbone.

The rest of the paper is organized as follows. Section II reports related work. Section III introduces our methods. Section IV analyzes the SFA method. Section V verifies the effectiveness and efficiency of our methods on static and neuromorphic vision datasets. Section VI discusses the

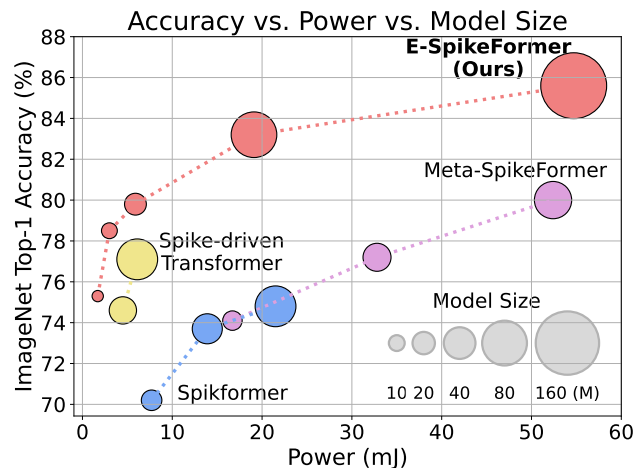


Fig. 1: E-SpikeFormer versus other spiking Transformers on ImageNet-1k at 224² input spatial resolution.

implementation of the proposed methods on neuromorphic chips. Section VII concludes this work.

II. PRELIMINARIES

Efficient Training in SNNs. Complex spatio-temporal dynamics make the training cost of SNNs naturally larger than that of ANNs. The multi-timestep simulation requires a memory of $\mathcal{O}(L \times T)$ for STBP training, where L is the layer and T is the timestep. The underlying logic of efficient SNN training is to decouple training memory from timestep T . For example, based on the gradient importance analysis, only the spatial gradients of SNNs are kept [36], [37]; pre-training is performed on a single timestep and then the model is fine-tuned to multiple timesteps [28], [38]; temporal or spatial reversible design to drop memory usage [39], [40]; the timestep during training and inference is dynamically reduced based on sample difficulty [41], [42]. These ideas are technically effective in alleviating the training challenges of large-scale SNNs but ignore a fundamental question: “why do SNNs need to employ multi-timestep simulation in static tasks?” In this work, we provide an in-depth analysis of this problem and provide a solution.

Architecture Design in SNNs. The main theme of SNNs in the deep learning era is scaling up. Architecture design is one of the important technical routes to reach this goal. Early efforts centered around the Spiking ResNet design. Directly copying the residual connection from ResNet [43] into SNNs [18] proved to be incapable of achieving identity mapping [44], and thus still suffered from performance degradation. To address this issue, SEW-ResNet [17] and MS-ResNet [19] successfully extend the depth of SNNs to several hundred layers by creating shortcuts between spikes and membrane potentials between different layers, respectively. In contrast, recent efforts to combine SNN and Transformer [45], [46] have mainly focused on how to design spike-driven self-attention mechanisms. None of the earlier spiking Transformers [47]–[49] can be spike-driven. It was not until Spike-driven Transformer [29] implemented spike-driven self-attention using

only addition and mask operations, thus making only sparse addition in spiking Transformer for the first time. Spike-driven Transformer v2 (i.e., Meta-SpikeFormer [28]) explores the meta-design [50], [51] of SNNs and discusses its potential impact on neuromorphic chip designs. In this work, we further optimize the redundancy design in the Meta-SpikeFormer to achieve higher performance with fewer parameters and power.

Masked Image Modeling (MIM) [30] is a self-supervised learning strategy that can effectively increase the size of visual models. Inspired by the masked pretrain-finetune language modeling in NLP [52], BEiT [53] first led the way in integrating MIM into the visual domain by training to predict visual tokens. Then, the famous MAE [30] introduced an asymmetric encoder and decoder architecture, bypassing masked tokens in computation-intensive encoders, while all tokens are handled through a lightweight decoder. Pre-training with MAE-style facilitates the Vision Transformer (ViT) in bolstering its capacity for local bias and overcoming its performance degradation [30], [54]. MAE-style pre-training can also help CNN or CNN+ViT hybrid architectures scale up but requires adjustments to the training strategy. For example, MC-MAE [55], Convnext v2 [56], and Spark [31] have employed masked or sparse convolutions to prevent information leakage and reconcile disparities between pre-training and fine-tuning. Since we follow the meta-architecture design of SNN in [28], and the meta-SNN is in the hybrid form of Conv+ViT, we draw on the hybrid MIM framework in [31], [55], [56] to realize large-scale pre-training of our E-SpikeFormer.

III. METHODS

We introduce and analyze the spiking neuron models in Section III-A. Then we propose the new training method, E-SpikeFormer architecture, and MIM strategy for scaling SNNs in Section III-B, III-C, and III-D, respectively.

A. Spiking Neuron

1) *Integration, Firing and Reset Mechanisms:* The spatio-temporal dynamics and spike communication of spiking neurons are abstracted from biological neurons [3]. Due to the high computational complexity of biological neuron dynamics, existing spiking neurons are simplified into differential equations to facilitate computer simulations. By solving the differential equation, an iterative version of the spiking neuron can be obtained [15], [57]:

$$\mathbf{U}[t] = \beta \mathbf{H}[t-1] + \mathbf{X}[t], \quad (1)$$

$$\mathbf{S}[t] = \text{Hea}(\mathbf{U}[t] - V_{\text{th}}), \quad (2)$$

$$\mathbf{H}[t] = \begin{cases} \mathbf{U}[t] \cdot (1 - \mathbf{S}[t]) + V_{\text{reset}} \cdot \mathbf{S}[t], & \text{hard reset,} \\ \mathbf{U}[t] - V_{\text{th}} \cdot \mathbf{S}[t], & \text{soft reset,} \end{cases} \quad (3)$$

where t is the timestep, $\mathbf{U}[t]$ represents the membrane potential after charging but before firing. Spatial input $\mathbf{X}[t]$ is extracted from the original spike input through a Conv or MLP operation, temporal input $\beta \mathbf{H}[t-1]$ is derived from the decay of the membrane potential at the previous timestep, β is the decay (leaky) factor, $\mathbf{S}[t]$ is the output spike, $\mathbf{H}[t]$ is the membrane potential after firing, $\text{Hea}(\cdot)$ is the Heaviside function that

$\text{Hea}(x) = 1$ if $x \geq 0$. When $\mathbf{U}[t]$ exceeds the threshold V_{th} , spiking neurons will fire spikes, and the membrane potential is reset to V_{reset} .

Eq. (1), Eq. (2), Eq. (3) represent the integration, firing, and reset mechanisms of spiking neurons, respectively:

i) Integration defines how spatio-temporal information is fused. For example, when using hard reset, in Integrate-and-Fire (IF) neurons [58], $\beta = 1$; whereas in Leaky-Integrate-and-Fire (LIF) [3], $\beta \neq 1$. Thus, LIF and IF memorize information from previous timesteps differently.

ii) Firing defines how information is transmitted in SNNs. Spiking neurons fire binary spikes only when the membrane potential exceeds the threshold. SNNs then can enjoy the low-power nature brought by event (spike)-driven, which means that the network only triggers sparse addition when there is input [29], [59].

iii) Reset defines how a spiking neuron memorizes and forgets (Fig. 2). We discuss three reset mechanisms: no reset [60], hard reset [3], soft reset [58]. No reset and hard reset represent two extreme cases. The former remembers all information, resulting in infinite firing over time. The latter forgets any existing information after a neuron fires a spike (if $V_{\text{reset}} = 0$). As a trade-off between no reset and hard reset, soft reset preserves some information while subtracting V_{th} from the membrane potential after firing a spike.

2) *Inherent Flaws in Binary Spike Firing:* Spiking neuron models are typically defined by their integration and reset mechanisms. Based on Eq. (1) and Eq. (3), four types of spiking neurons can be identified: LIF with Hard Reset (LIF-HR); LIF with Soft Reset (LIF-SR); IF with Hard Reset (IF-HR); IF with Soft Reset (IF-SR).

An often overlooked fact is that the firing mechanism (Eq. (2)) also influences the integration and reset mechanisms. Consequently, binary spike firing introduces at least two intrinsic flaws for spiking neurons:

i) **Spatial Representation.** It is impossible to assess the importance of the current spatial input. Regardless of how large the membrane potential is, the output is just a spike. This can also be interpreted as a large quantization error in converting the membrane potential to a spike.

ii) **Temporal Dynamics.** It is impossible to perform data-dependent memorization or forgetting. As shown in Eq. (3), $\mathbf{S}[t]$ can only take on the value 0 or 1, implying that the soft reset can only forget fixed values.

Let's take the spiking neurons in Fig. 2 as an example. Assume that the spatial input at $(t-1)$ -th is very important (large value). In the case of the hard reset, the spiking neuron outputs a spike and clears the membrane potential to V_{reset} . Obviously, this has a large quantization error and cannot reflect the fact that the input is very important. In the case of the soft reset, the spiking neuron outputs a spike, but the membrane potential passed to the next timestep is still large. One possibility is that the spiking neuron fires for a long time afterward, and then you cannot tell whether the firing at t -th is caused by the $(t-1)$ -th spatial input or by the current spatial input (as shown in the example in Fig. 2(d)). Thus, the soft reset may result in identical memory and forgetting processing at each timestep.

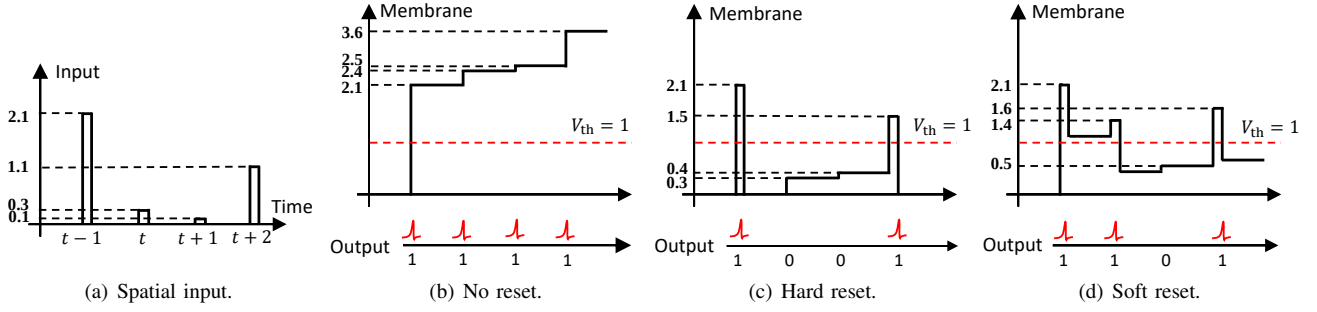


Fig. 2: The impact of the reset mechanism on the spatio-temporal dynamics of spiking neurons. (a) The $(t-1)$ -th and $(t+2)$ -th inputs are greater than the threshold, the t -th and $(t+1)$ -th inputs are not. Here we set $\beta = 1$. (b) If no reset, the spiking neuron will keep firing after the input exceeds the threshold at some point. (c) If hard reset to zero, a spike will be fired whenever the spatial input at the current timestep exceeds the threshold. Inputs that do not exceed the threshold will not trigger a spike. (d) If there is a soft reset, the membrane potential at t -th timestep contains part of the information of $(t-1)$ -th timestep; even if the input at t -th is small, a spike may be fired.

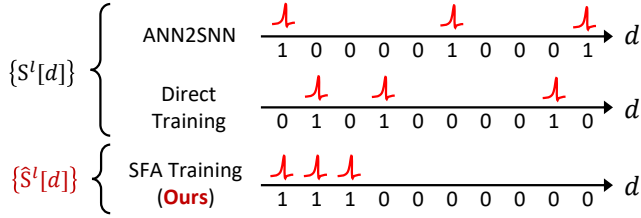


Fig. 3: Spike firing patterns. Assuming an approximation of 0.3 ($a_D^l = 0.3$), i.e., three spikes need to be fired at ten timesteps ($D = 10$). The timesteps at which the three spikes appear are synchronized and randomized for ANN2SNN and direct training SNN, where synchronization means that all ten timesteps must be completed to compute the spike firing rate. In contrast, SFA training will not fire spikes on the remaining seven timesteps after firing them on the first three timesteps. Therefore, the spike firing in SFA can be realized in an asynchronous manner (detailed in Section VI-A).

B. Spike Firing Approximation (SFA) Method

Previous SNNs exploited multi-bit spikes (integers) [61], [62] or continuous value [63] for the motivation of reducing quantization error. This in fact mitigates the flaws of binary spikes. But, the concern that comes with it is that the spike-driven nature is destroyed. We address these concerns via three steps: Step-1, we assume that the signal transmitted between spiking neurons is the spike firing rate. Step-2, during training, we approximate and replace the above spike firing rate with integer-valued activations as the forward-propagation signal. Step-3, during inference, we preserve the spike-driven nature by equating the integer activation to binary spikes over multi-timesteps. We name the whole process “Spike Firing Approximation (SFA)”.

SFA Training and Inference. Existing ANN2SNNs [13] and vanilla direct training SNNs [64], [65] can be understood as rate coding (Fig. 3). Inspired by this, we assume that spiking neurons transmit spike firing rates (continuous values) in the spatial dimension.

In Step-1, The spike firing rate a_D^l of a spiking neuron is defined as the ratio of fired spikes to the total timestep:

$$a_D^l = \frac{1}{D} \sum_{d=1}^D \mathbf{S}^l[d], \quad (4)$$

where $\{\mathbf{S}^l[d]\}_D$ is the spike train in ANN2SNN or direct training over given D timesteps at l -th layer.

In Step-2, we perform integer activation training, i.e., employ an integer to approximate a_D^l . We replace $\sum_{d=1}^D \mathbf{S}^l[d]$ with the integer-valued activation of a single timestep:

$$\mathbf{S}_D^l = \text{Fire}_D(\mathbf{U}^l) = \lfloor \text{clip}\{\mathbf{U}^l, 0, D\} \rfloor, \quad (5)$$

where $\mathbf{S}_D^l$ is the integer activation within $[0, D]$, $\mathbf{U}^l$ is the membrane potential before firing, $\text{Fire}_D(\cdot)$ is the integer-valued fire function, D indicates the maximum integer value allowed to be fired. And $\text{clip}\{\mathbf{U}^l, 0, D\}$ confines the value of $\mathbf{U}^l$ within 0 and D , $\lfloor \cdot \rfloor$ rounds to the nearest integer.

In Step-3, we perform spike activation inference:

$$\mathbf{S}_D^l = \sum_{d=1}^D \hat{\mathbf{S}}^l[d], \quad (6)$$

where $\{\hat{\mathbf{S}}^l[d]\}_D$ is the spike train in our SFA training. The spatial input to the spiking neuron at $(l+1)$ -th layer is

$$\mathbf{X}^{l+1} = \mathbf{W}^{l+1} a_D^l = \mathbf{W}^{l+1} \frac{1}{D} \mathbf{S}_D^l = \left(\frac{1}{D} \mathbf{W}^{l+1} \right) \sum_{d=1}^D \hat{\mathbf{S}}^l[d]. \quad (7)$$

As a simple summary, the SFA method replaces $\mathbf{S}[t]$ in Eq. (2) with $\mathbf{S}_D^l$ during training; when inference, each timestep is expanded into $1 \times D$ timesteps, and the corresponding spike train is $\{\hat{\mathbf{S}}^l[d]\}_D$. The integer firing of SFA not only reduces the quantization error when converting the membrane potential of spiking neurons into spikes, but also enhances the forgetting capability of spiking neurons at the current timestep to address temporal dynamics deficiencies. More details of SFA method are given in Section IV. In addition, in Section V, we systematically analyze that replacing the existing $\{\mathbf{S}^l[d]\}_D$ with our $\{\hat{\mathbf{S}}^l[d]\}_D$ spike firing pattern (Fig. 3) has a profound

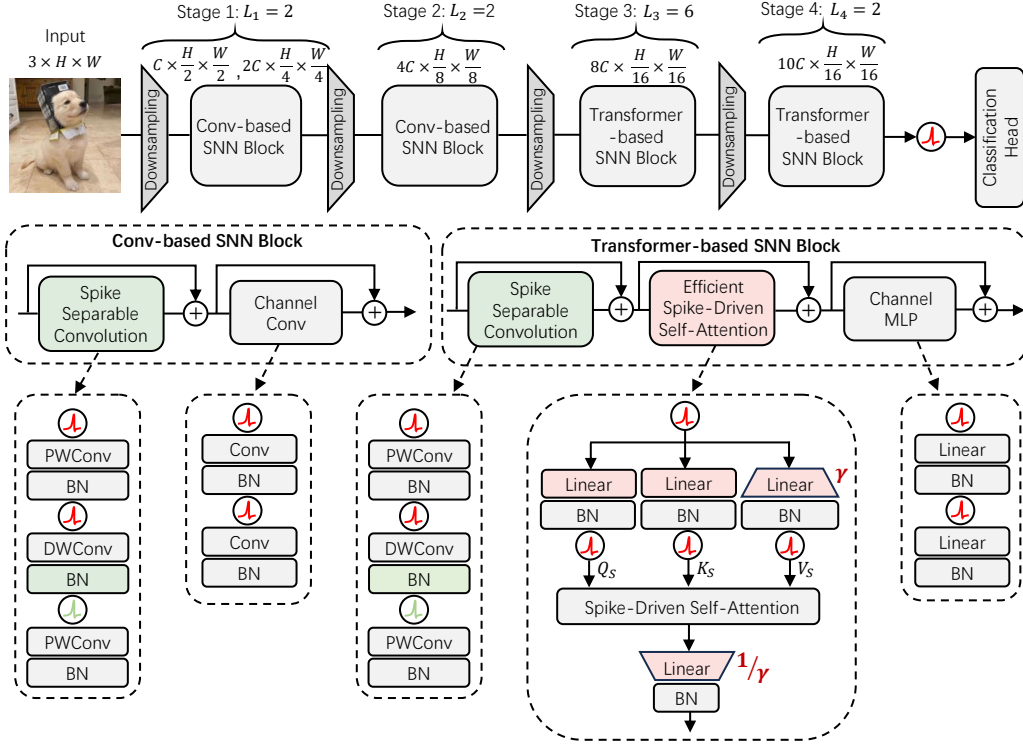


Fig. 4: The overview of E-SpikeFormer. In general, we follow the design of Meta-SpikeFormer [28] on a macro level, such as CNN-based and Transformer-based SNN blocks and their proportions at each stage. We redesign the interior of CNN-based and Transformer SNN blocks with the goal of efficient architecture. In this figure, we have marked all upgraded designs in *green* or *pink*. In the green part, we insert a BN layer and a spiking neuron layer after the DWConv layer in the CNN-based block to avoid the increase in power caused by the larger convolution kernel size after re-parameterization. In the pink part, the Re-parameterization Convolution (RepConv) in [28] is replaced by a linear layer to reduce power, and we compensate for the potential performance loss by expanding the channel number of V_S . In addition, we add a Spike Separable Convolution (SpikeSepConv) module to the Transformer-based SNN block in [28], which helps performance.

impact on the training, performance, energy consumption, and scaling of SNNs.

C. E-SpikeFormer

Meta-SpikeFormer [28] explores the meta-design of SNNs in Spike-Driven Self-Attention (SDSA), architecture, and shortcut. We here focus on designing a more efficient SNN based on Meta-SpikeFormer, and name it "E-SpikeFormer".

Overview. E-SpikeFormer contains Conv-based and Transformer-based SNN stages (Fig. 4). Each stage contains several blocks, each block containing a token mixer module and a channel mixer module. Many Reparameterization Convolutions (RepConv) are used in Meta-SpikeFormer, which results in a significant increase in power. According to Table 5 in Meta-SpikeFormer, the RepConv part accounts for approximately 60% of the total energy cost. In this work, we eliminate all RepConvs and compensate for performance by incorporating a set of additional lightweight Spike Separable Convolution (SpikeSepConv). In Fig. 4, we label all upgraded designs of our E-SpikeFormer relative to Meta-SpikeFormer as *green* or *pink*. We see that there are three primary modifications in our approach: i) In the Conv-based SNN block, the token mixer has been replaced

with a SpikeSepConv; ii) An SpikeSepConv module has been added prior to the token mixer (i.e., SDSA module) in the Transformer-based SNN block; iii) In the SDSA module, RepConv has been substituted with a linear operation.

Conv-based SNN Block in [28] uses separable convolution [66] as the token mixer. However, in SNNs, the direct connection between depthwise and pointwise convolutions with subsequent reparameterization poses a serious energy consumption problem. Our solution is to add a Batch Normalization (BN) layer [67] and a spiking neuron layer between depthwise and pointwise convolutions. Specifically, our CNN-based SNN block is written as:

$$\mathbf{U}' = \mathbf{U} + \text{SpikeSepConv}(\mathbf{U}), \quad (8)$$

$$\mathbf{U}'' = \mathbf{U}' + \text{ChannelConv}(\mathbf{U}'), \quad (9)$$

$$\text{SpikeSepConv}(\mathbf{U}) = \text{Conv}_{\text{pw2}}(\text{SN}(\text{Conv}_{\text{dw}}(\text{SN}(\text{Conv}_{\text{pw1}}(\text{SN}(\mathbf{U})))))), \quad (10)$$

$$\text{ChannelConv}(\mathbf{U}') = \text{Conv}(\text{SN}(\text{Conv}(\text{SN}(\mathbf{U}')))), \quad (11)$$

where $\text{SpikeSepConv}(\cdot)$ is the token mixer, $\text{ChannelConv}(\cdot)$ is the channel mixer, $\text{Conv}_{\text{pw1}}(\cdot)$ and $\text{Conv}_{\text{pw2}}(\cdot)$ are pointwise convolutions, $\text{Conv}_{\text{dw}}(\cdot)$ is depthwise convolution [68], $\text{Conv}(\cdot)$ is the vanilla convolution. $\text{SN}(\cdot)$ is the spike neuron layer. Note, we ignore the BN layer for ease of writing.

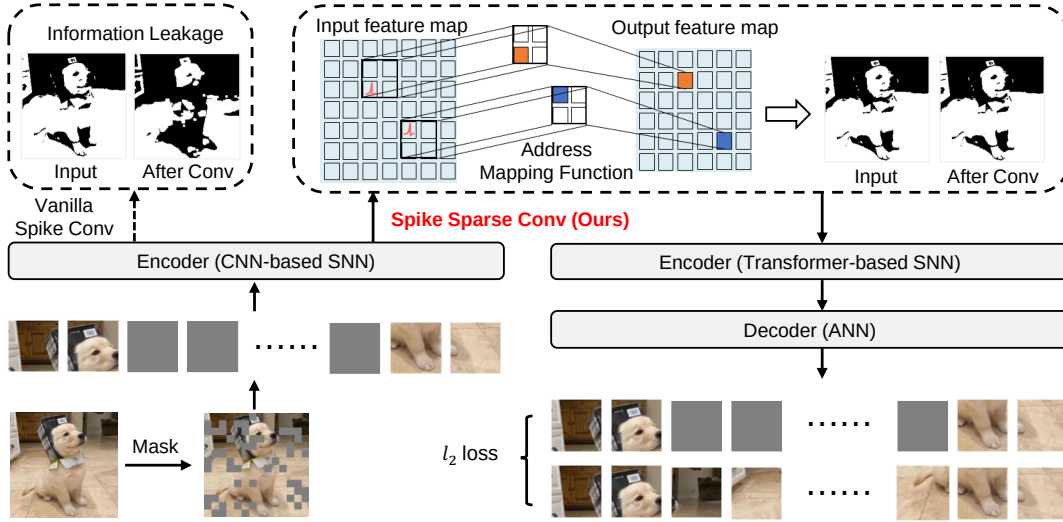


Fig. 5: The overview of MIM pre-train in E-SpikeFormer. It consists of a SNN encoder and a ANN decoder. The encoder processes only the visible pixels. The decoder reconstructs the images to learn representations, which will be removed in the fine-tuning stage. Using Vanilla Spike Convolution (VSC) as an encoder will lead to information leakage, so we designed Spike Sparse Convolution (SSC) to avoid this flaw. White and black in the figure represent the one/zero area respectively. Exploiting VSC will cause the black zero area to continue to grow as the depth increases until there is no valid information. We show that SSC is a natural fit for neuromorphic chips due to the spike-driven nature of SNNs (Section VI-B).

Transformer-based SNN Block. Motivated by the observation that adding a convolution layer before the self-attention operator to extract local features enhances the integration of global features [69], [70], we introduce a lightweight SSC module before the SDSA. This addition aims to offset the performance decline resulting from the elimination of RepConv. Then, the Transformer-based SNN Block can be written as

$$\begin{aligned} \mathbf{U}' &= \mathbf{U} + \text{SpikeSepConv}(\mathbf{U}), \\ \mathbf{U}'' &= \mathbf{U}' + \text{E-SDSA}(\mathbf{U}'), \\ \mathbf{U}''' &= \mathbf{U}'' + \text{ChannelMLP}(\mathbf{U}''), \end{aligned}$$

where E-SDSA($\cdot$) is the proposed E-SDSA module.

Efficient Spike-Driven Self-Attention (E-SDSA) Module. In this work, we combine the generation of $\mathbf{K}_S$, $\mathbf{Q}_S$, $\mathbf{V}_S$, the SDSA operator, and the subsequent linear operator as the E-SDSA module (pink module in Fig. 4). In E-SDSA, we utilize linear operations to generate $\mathbf{K}_S$, $\mathbf{Q}_S$, $\mathbf{V}_S$ instead of energy-consuming reparameterization convolutions. We adopt the iRMB technique in [71] that expands the channels of $\mathbf{V}_S$ by a factor of γ to enhance the representation of E-SDSA. Specifically, the E-SDSA module can be written as

$$\begin{aligned} \mathbf{Q}_S &= \text{SN}(\text{Linear}(\mathbf{U})), \mathbf{K}_S = \text{SN}(\text{Linear}(\mathbf{U})), \\ \mathbf{V}_S &= \text{SN}(\text{Linear}_\gamma(\mathbf{U})), \\ \mathbf{U}' &= \text{Linear}_{\frac{1}{\gamma}}(\text{SN}(\mathbf{Q}_S \mathbf{K}_S^T \mathbf{V}_S * \text{scale})), \end{aligned}$$

where $\text{Linear}(\cdot)$ is the linear operator with learnable matrix $\mathbf{W} \in \mathbb{R}^{C_{in} \times C_{out}}$ and bias $\mathbf{b} \in \mathbb{R}^{C_{out}}$ (C_{in} and C_{out} are the dimension of input and output respectively). Its mathematics form can be written as $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$. Similarly, $\text{Linear}_\gamma(\cdot)$ has weight of $\mathbf{W}_\gamma \in \mathbb{R}^{C_{in} \times \gamma C_{out}}$ and bias of $\mathbf{b}_\gamma \in \mathbb{R}^{\gamma C_{out}}$. $\text{Linear}_{\frac{1}{\gamma}}(\cdot)$ has weight of $\mathbf{W}_{\frac{1}{\gamma}} \in \mathbb{R}^{\gamma C_{out} \times C_{in}}$ and bias of

$\mathbf{b}_{\frac{1}{\gamma}} \in \mathbb{R}^{C_{in}}$. $\text{SN}(\mathbf{Q}_S \mathbf{K}_S^T \mathbf{V}_S * \text{scale})$ is the SDSA operator, which is consistent with Meta-SpikeFormer [28], and constant scale factor can be reparameterized into SN's threshold.

D. Spike Sparse Convolution for Scaling Up SNNs

Directly scaling the ViT leads to performance degradation [46], [72]. The proposed E-SpikeFormer was not immune to this problem. The pretrain-finetune MIM paradigm is one of the common methods to solve the degradation problem [30]. Tian *et al.* [31] pointed out that the MIM strategy cannot be used directly in CNNs. An input image is divided into non-overlapping patches in ViTs. Benefiting from ViT's ability to handle irregular and non-overlapping patches, MIM can simply drop masked patches to remove the information. In contrast, CNNs not only operate on regular grids but also perform sliding windows with overlapping. Directly employing MIM in CNNs would result in *information leakage*. That is, since the input contains lots of blank (masked) areas, deep convolutions can hardly learn any features as the network deepens. Tian *et al.* proposed a solution SparK [31] that involves encoding the input image after applying a zero-mask, using the sparse convolution encoder.

The proposed MIM-SNN follows SparK (Fig. 5) but with a newly designed Spike Sparse Convolution (SSC) encoder. The overall process can be written as $G = \mathcal{F}_E \circ \mathcal{F}_D$, which embodies an encoder-decoder architecture. An encoder $\mathcal{F}_E$ is responsible for mapping masked inputs $\mathbf{X}_1$ to a latent feature $\mathbf{Z} = \mathcal{F}_E(\mathbf{X}_1)$. A decoder $\mathcal{F}_D$ reverses this process, mapping $\mathbf{Z}$ back to the pixel space to reconstruct the target view $\mathbf{X}_2$ ($\mathbf{X}_1$ and $\mathbf{X}_2$ are complementary).

Masking. Patch-wise masking is the first step of MIM. An image $\mathbf{X}$ is restructured into a succession of flattened 2D patches, $\mathbf{X} \in \mathbb{R}^{r \times p}$, where p denotes the patch size. Then

a random binary mask $m \in \{0, 1\}^n$ is generated, enabling the acquisition of two complementary [73] masked views of $\mathbf{X}$:

$$\mathbf{X}_1 = \mathbf{X}[n] \in \mathbb{R}^{r_1 \times p}, \quad \mathbf{X}_2 = \mathbf{X}[1 - n] \in \mathbb{R}^{r_2 \times p}, \quad (12)$$

r_1 and r_2 are integers that $r_1 = n(1 - \mu)$ and $r_2 = n\mu$, ensuring that their sum equals n , μ is the masking ratio.

Encoder. The masked image is similar to neuromorphic data, which only has events when the brightness changes and therefore contains large blank (zero) areas. Intuitively, convolutions in SNNs are naturally sparse convolutions due to their spike-driven nature. As shown in Fig. 5, according to the address of the input spike, the address mapping function [9] outputs the address of the neuron and synapse that need to perform sparse synaptic addition operations. Unfortunately, using Vanilla Spike Convolution (VSC) cannot avoid information leakage in MIM-SNN. This discrepancy arises from the fact that the VSC would perform convolution at all positions, eventually causing information leakage (it can also be called spike degradation [19], the spiking neurons in deep layers almost no longer fire spikes).

To address this challenge, we propose the SSC, which only performs convolution on positions that are not masked. We reshape $\mathbf{X}_1$ with C_{in} channels with height H and width W in the d dimensional spatial space. We process this feature by SSC with kernel weights $\mathbf{W} \in \mathbb{R}^{K^d \times C_{in} \times C_{out}}$. As an example, in the 2D coordinate space, $\mathbf{W}$ contains $C_{in} \times C_{out}$ channels with K kernel size and $K^d=3^2$. The convolution process of SSC is represented as

$$\mathbf{Y}_p = \sum_{k \in K_d} \alpha(\mathbf{W}_k \cdot \mathbf{X}_1^{p_k}), \quad (13)$$

where k enumerates all discrete locations in the kernel space K^d . $p_k = p + k$ is the corresponding location around center p , where k is an offset distance from p . $\alpha \in \{0, 1\}$ is a selector. When the center $p \in \mathbf{X}_1$ is the masked area, α is 0, indicating that the point p does not participate in the computation, preserving the masked feature map. $\alpha = 1$ denotes that the region is unmasked and active for computation. During pre-training, the encoder exploits SSC; during fine-tuning, the weights are converted back to VSC.

Decoder. The output of the encoder is supplied to the decoder along with learnable masked tokens. Then the masked tokens are reconstructed into an image. There are usually two options for the decoder: plain [55] and multi-scale [31] ANN. We found that different decoders have less impact on the accuracy of MIM-SNN pre-train. So during pre-training, we employed a plain ANN (Transformer) as a decoder, which is removed in the fine-tuning and would not impact SNNs' spike-driven inference.

Loss Function. In pre-training, we use MSE loss as the metric function $\mathcal{L}$ and normalized pixel values of each masked patch as the prediction target.

$$\mathcal{L}_G = \mathbb{E}_{\mathbf{X}} \mathbb{E}_{\mathbf{X}_1, \mathbf{X}_2 | \mathbf{X}} \|\mathcal{F}_D(\mathcal{F}_E(\mathbf{X}_1)) - \mathbf{X}_2\|^2, \quad (14)$$

where $\mathcal{F}_D(\mathcal{F}_E(\mathbf{X}_1))$ is the decoder output and $\|\cdot\|^2$ is the l_2 norm. Then we exploit the distillation [72] technique during fine-tuning, which is the same in Meta-SpikeFormer [28].

Analysis on MIM-SNN. In Section V-D, we reveal how binary spiking affects the scaling of SNNs. In Section VI-B, we show that the difference between SSC and our VSC lies in the address mapping function [9], thus neuromorphic chips are naturally capable of performing SSC-based MIM-SNN pre-train.

IV. ANALYSIS OF SPIKE FIRING APPROXIMATION

First, we demonstrate the equivalence between integer-based training and the corresponding spike train inference within the SFA method. Subsequently, we analyze the approximation error that arises from substituting the spike firing rate with integer activations.

A. Implementation of Spike-driven Inference

The transition between integer training and spike inference can be losslessly achieved by IF-SR neurons with $V_{th} = 1$.

Proposition 1. Consider the $\text{Fire}_D(\cdot)$ function at l -th layer in SNN, its integer-value output is equal to the sum of spikes generated by IF-SR spiking neuron with D timesteps:

$$\mathbf{S}_D^l = \sum_{d=1}^D \hat{\mathbf{S}}^l[d], \quad (15)$$

where $\mathbf{S}_D^l$ is the integer value fired by $\text{Fire}_D(\cdot)$ at l -th layer and $\{\hat{\mathbf{S}}^l[d]\}_D$ is the spike train generated by IF-SR spiking neuron over given D timesteps with $V_{th} = 1$.

Proof. Feeding the membrane potential input of Eq. (5) into an IF-SR neuron with $V_{th} = 1$, the spike train generated is:

$$\{\hat{\mathbf{S}}^l[d]\}_D = \text{IF-SR}(\mathbf{U}^l), \quad (16)$$

which means that the IF-SR spiking neuron subtracts 1 at each timestep until the membrane potential is less than the threshold. The underlying assumption is that there is non-zero input only at timestep $d = 1$, and the input is zero at timesteps $d = 2, \dots, D$. Then the sum of the spike train is:

$$\sum_{d=1}^D \hat{\mathbf{S}}^l[d] = \lfloor \text{clip}\{\mathbf{U}^l, 0, D\} \rfloor, \quad (17)$$

where $\lfloor \cdot \rfloor$ is the floor function. According to Eq. (5), we have:

$$\mathbf{S}_D^l = \lfloor \text{clip}\{\mathbf{U}^l, 0, D\} \rfloor, \quad (18)$$

$$= \lfloor \text{clip}\{\mathbf{U}^l + 0.5, 0, D\} \rfloor, \quad (19)$$

$$= \lfloor \text{clip}\{\mathbf{U}^l, 0, D\} \rfloor, \quad (20)$$

where in Eq. (20), the bias '0.5' about the equivalence between the round function $\lfloor \cdot \rfloor$ and floor function $\lfloor \cdot \rfloor$ can be incorporated into the weight. Combining Eq. (17) and Eq. (20), we have:

$$\mathbf{S}_D^l = \lfloor \text{clip}\{\mathbf{U}^l, 0, D\} \rfloor = \sum_{d=1}^D \hat{\mathbf{S}}^l[d]. \quad (21)$$

□

Eq. (21) implies the equivalence from integer training to spike train inference. We analyze how this affects training, depending on the task type.

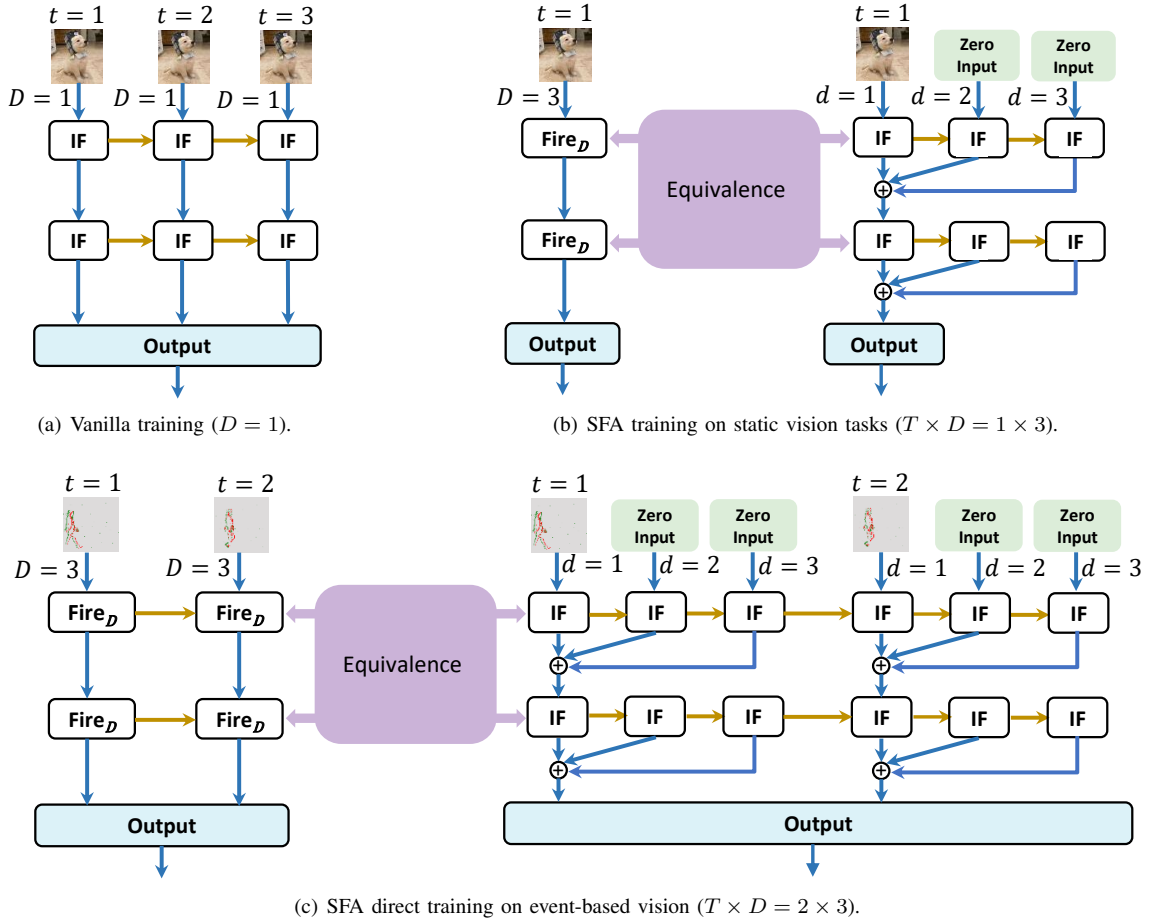


Fig. 6: Comparison of vanilla and SFA training in SNNs. T in SNNs typically denotes the timestep. For vanilla training SNNs, T remains consistent between training and inference. In SFA training, the timestep during training is T , but it is expanded to $T \times D$ timesteps during inference (The neuron in the picture is IF-SR). Since the activations in SFA training are integers, another meaning of D is the upper limit of the integer values allowed for activation during training. In the context of vanilla training SNNs, D is equal to 1. (a) Direct training uses the same processing approach for both static and dynamic vision tasks. Since $D = 1$, direct training will not be able to avoid the mechanism defects of binary spike firing in dynamic vision tasks. (b) For static tasks, the SFA method has advantages in training cost (one-timestep training) and inference energy cost (the image is input only once). (c) For dynamic tasks, the SFA method can simultaneously mitigate the flaws of spiking neurons in both spatial representation and temporal dynamics by expanding the timestep.

One-timestep SFA training for static vision tasks. Static vision tasks, such as classification and detection, do not contain temporal information. As shown in Fig. 6(a), to overcome the defect of spiking neurons in spatial representation, previous direct training forcibly constructed multi-timestep rate encoding through repeated input. Since SFA training changes the spike firing pattern, for static tasks we only need one-timestep training and then convert it to multi-timestep spike-driven inference (Fig. 6(b)). SFA training has two obvious advantages over vanilla training: First, the one-timestep training cost is lower; Second, during inference, there is no need to repeatedly input images in SFA.

Multiple-timesteps SFA training for dynamic vision tasks. For temporal dynamic vision tasks, multi-timestep simulation cannot be understood as rate encoding, but the same processing as static tasks is still used in direct training. Therefore, in dynamic vision tasks, the spatial representation

and temporal dynamics defects of the binary spike firing will erupt simultaneously in direct training.

SFA training overcomes these two flaws by expanding the timestep to $T \times D$ during inference: First, integer values reflect the importance of spatial inputs; Second, the size of the integers is determined by the spatial inputs, and thus is a data-dependent mechanism, equivalent to the fact that the spiking neurons can be dynamically forgetting (soft reset). In summary, each timestep in SFA training is expanded into $1 \times D$ timesteps, which can be viewed as executing two reset mechanisms simultaneously. When the timestep is within the range of $[(t-1) \times D + 1, t \times D]$, IF-SR neurons are used. When the timestep is at $[1, \dots, (t-1) \times D, t \times D, \dots, T \times D]$, LIF-HR/LIF-SR/IF-HR/IF-SR dynamic can be applied. We will discuss how to implement this hybrid spiking neuron mechanism in the neuromorphic chip in Section VI-A.

B. Approximation Error Analysis

Using finite integer values to approximate the continuous spike firing rate introduces an approximation error. The spike firing rate of the l -th layer over D timesteps is:

$$a_D^l = \frac{1}{D} \mathbf{S}_D^l = \frac{1}{D} \text{Fire}_D(\mathbf{U}^l) = \frac{1}{D} [\text{clip}\{\mathbf{U}^l, 0, D\}]. \quad (22)$$

Definition 1 (Forward Approximation Error). *Given the continuous input membrane potential at l -layer, denoted as $\mathbf{U}^l[d]$, we define the approximation error as follows:*

$$\text{Err}^l = \text{ReLU}(\mathbf{U}^l[d]) - \mathcal{F}(\mathbf{U}^l[d]), \quad (23)$$

where $\text{ReLU}(\cdot)$ represents the ReLU function and $\mathcal{F}(\cdot)$ is the approximation function. In the ideal scenario $\mathcal{F}(\mathbf{U}^l[d]) = \text{ReLU}(\mathbf{U}^l[d])$, the approximation error equals to zero.

Since the approximation function in SFA is $\mathcal{F}(\cdot) = \frac{1}{D} \text{Fire}_D(\cdot)$, according to the Definition 1, the approximation error is :

$$\text{Err}_{\text{SFA}}^l = \text{ReLU}(\mathbf{U}^l[d]) - \frac{1}{D} [\text{clip}\{\mathbf{U}^l[d], 0, D\}]. \quad (24)$$

Continuing with the following deductions:

$$\text{Err}_{\text{SFA}}^l = \begin{cases} 0, & \text{if } \mathbf{U}^l[d] < 0 \\ \mathbf{U}^l[d] - \frac{1}{D} [\mathbf{U}^l[d]], & \text{if } 0 \leq \mathbf{U}^l[d] < D \\ \mathbf{U}^l[d] - 1, & \text{if } \mathbf{U}^l[d] \geq D \end{cases} \quad (25)$$

In the backpropagation stage, the SFA (Eq. (5)) is non-differentiable. Previous studies have introduced various surrogate gradient functions [15], [25], primarily designed to address binary spike outputs. In our approach, we consistently utilize rectangular windows as the surrogate function. For simplicity, we retain gradients only for neurons activated within the $[0, D]$ range, setting all others to zero.

Using rectangular windows as the surrogate function cannot avoid gradient error. We define this gradient error as the difference between the integral of the surrogate gradient function and the forward approximation function of SFA, which can be expressed as follows:

Definition 2 (Backward Gradient Error). *Given the continuous input membrane potential at l -layer, denoted as $\mathbf{U}^l[d]$, we define the gradient error as follows:*

$$\text{Err}^l = \int_{-\infty}^{\infty} \text{Rect}_{[0,D]}(\mathbf{U}^l[d]) - \mathcal{F}(\mathbf{U}^l[d]), \quad (26)$$

where $\text{Rect}_{[0,D]}(\cdot)$ is rectangular windows between 0 and D . $\mathcal{F}(\cdot)$ is the approximation function. In the ideal scenario, $\nabla \mathcal{F}(\mathbf{U}^l[d]) = \text{Rect}_{[0,D]}(\mathbf{U}^l[d])$, the gradient error equals zero.

According to the Definition 2, the gradient error is:

$$\text{Err}_{\text{SFA}}^l = \int_{-\infty}^{\infty} \text{Rect}_{[0,D]}(\mathbf{U}^l[d]) - \frac{1}{D} [\text{clip}\{\mathbf{U}^l[d], 0, D\}]. \quad (27)$$

Continuing with the following deductions:

$$\text{Err}_{\text{SFA}}^l = \begin{cases} 0, & \text{if } \mathbf{U}^l[d] < 0 \\ \mathbf{U}^l[d] - \frac{1}{D} [\mathbf{U}^l[d]], & \text{if } 0 \leq \mathbf{U}^l[d] < D \\ 1, & \text{if } \mathbf{U}^l[d] \geq D \end{cases} \quad (28)$$

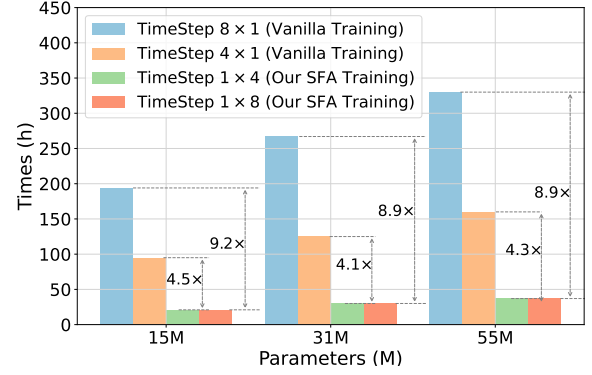


Fig. 7: Vanilla and SFA training time on the ImageNet with 200 epochs on 8 NVIDIA-A100-40G. On the static dataset, we use $T = 1$ during training and extend it to D timesteps during inference. Since we exploit one timestep training, the training cost is significantly reduced.

In this subsection, we examine the approximation and gradient errors introduced by the $\text{Fire}_D(\cdot)$ function in SFA as depicted in Equations (25) and (28). Our analysis reveals that larger values for the timestep D lead to reduced errors. It should be noted that large timesteps may boost latency and energy consumption.

V. EXPERIMENTS

In Section V-A, we report the main results of the proposed methods across various vision tasks. In Section V-B, we conduct ablation studies on SFA, E-SpikeFormer, and MIM pre-training. Sections V-C and Sections V-D analyze the impact of SFA on the performance, power, and scalability of SNNs.

A. Main Results

We evaluate our methods on various tasks, including image classification (ImageNet-1k [32]), object detection (COCO [33]), semantic segmentation (ADE20K [34]), neuromorphic (HAR-DVS [35], CIFAR10-DVS [89]) tasks. We follow the pre-training and fine-tuning approach in [28] to test E-SpikeFormer on COCO and ADE20K datasets. The model trained on ImageNet is used to fine-tune the detection or segmentation heads. Power evaluation methods and details of training/configurations are given in the Section S1 and Section S2 of Supplementary Materials, respectively.

1) *Image Classification*: The proposed methods are validated at 6 model scales (Table I) on ImageNet-1K [32], a prominent dataset with 1.3M training and 50K validation images across 1K classes.

Compared with prior SNN Baselines. We comprehensively compare our E-SpikeFormer with prior SNNs in accuracy, parameter, and power. E-SpikeFormer achieves SOTA with clear all-around advantages, e.g., **E-SpikeFormer** vs. Meta-SpikeFormer [28] vs. MS-ResNet [19]: Param, **19M** vs. 55M vs. 77M; Acc, **79.8%** vs. 80.0% vs. 76.3%; Power, **5.9mJ** vs. 52.4mJ vs. 10.2mJ. We have brought *qualitative* improvements to the SNNs' accuracy on ImageNet. In the small-scale 5M, 10M, and 19M models, we achieve 75.3%, 78.5%, and

TABLE I: Comparison with previous works on ImageNet-1k. The default inference input resolution is 224×224 . * The input crops are enlarged to 384×384 in inference. We rewrite the timesteps of all direct training SNNs into the format of $T \times D$, where T refers to the timestep and D is the upper limit of the integer activation during training. In previous SNNs, D defaulted to 1. Note, “Param” and “Step” in all Table headers of this paper denote “Parameters” and “Timestep”, and all “Step” involved in direct training SNNs are written in the form of $T \times D$ by default.

Methods	Architecture	Spike-driven	Param (M)	Power (mJ)	Step	Top-1 Acc.(%)
ANN	DeiT-S [72]	✗	22.0	21.2	1	79.8
	PVT-S [74]	✗	24.5	35.0	1	79.8
	Swin-T [75]	✗	29.0	20.7	1	81.3
	CAFormer-M36 [76]	✗	56.0	60.7	1	85.2
	T2T-ViT _t -24 [77]	✗	64.1	69.0	1	82.6
	RepLKNet-31B [78]	✗	79.0	70.4	1	83.5
	DeiT-B [72]	✗	86.0	80.5	1	81.8
	FocalNet-B (LRF) [79]	✗	88.7	70.8	1	83.9
	ConvNeXt-B [80]	✗	89.0	70.8	1	83.8
	Focal-B [81]	✗	89.8	73.6	1	83.8
	CrossFormer-L [82]	✗	92.0	74.1	1	84.0
ANN2SNN	ResNet-34 [11]	✓	21.8	-	250	61.5
	Optimal ANN2SNN [83]	✓	-	-	1024	74.3
	Tandem VGG-16 [13]	✓	-	-	16	73.0
	Fast-SNN [84]	✓	138.4	-	7	73.0
	Masked spiking Transformer [14]	✗	28.5	-	512	78.5
	Two-stage ANN2SNN [85]	✓	-	-	128	74.9
Direct training	SEW-ResNet [17]	✗	60.2	12.9	4×1	69.2
	TET-ResNet [86]	✗	21.8	-	4×1	68.0
	DSR-ResNet [87]	✗	11.2	-	50×1	67.7
	MS-ResNet [19]	✓	77.3	10.2	4×1	75.3
	Att-MS-ResNet [20]	✗	78.4	7.3	4×1	77.1 *
	Spikformer [49]	✗	66.3	21.5	4×1	74.8
	Spike-driven Transformer [29]	✓	66.3	6.1	4×1	76.3
	Spikformer v2 [88]	✗	172.7	25.6	4×1	82.4
		✓	15.1	16.7	4×1	74.1
	Meta-SpikeFormer [28]	✓	31.3	32.8	4×1	77.2
Direct training		✓	55.4	52.4	4×1	80.0
		✓	5.1	1.7	1×4	75.3
		✓	10.0	3.0	1×4	78.5
		✓	19.0	5.9	1×4	79.8
		✓	52.0	14.4	1×4	80.6
	E-SpikeFormer	✓	83.0	19.1	1×4	83.2
	(This Work)	✓	83.0	30.9	1×8	84.0
		✓	83.0	-	1×8	85.2 *
		✓	173.0	35.6	1×4	84.7
		✓	173.0	54.7	1×8	85.1
		✓	173.0	-	1×8	86.2 *

79.8%, respectively, far exceeding prior SNNs. We extend the size and performance upper limit of SNNs to 173M and **86.2%**. The targeted efficient architecture design significantly reduces the power of our model. In Section V-B, we place the relevant ablation studies. D in the SFA setting affects accuracy and power (Section V-C) and scale (Section V-D). Since we use one-timestep training on ImageNet, the training cost is also significantly reduced. As shown in Fig. 7, we obtain a training speedup of about $9 \times$ on various scales, and going from 1×4 to 1×8 does not result in an increase in training hours. In addition, we provide the GPU memory acceleration ratio of SFA method in Section S3.

Compared with ANN Baselines. The accuracy of E-SpikeFormer has been able to surpass or comparable to some classic ANNs while maintaining the energy efficiency advantage. For example, at the 80M scale, **E-SpikeFormer** vs. ConvNeXt-B [80] vs. CrossFormer-L [82]: Param, **83M** vs. 89M vs. 92M; Acc, **83.2%** vs. 83.8% vs. 84.0%; Power, **19.1 mJ** vs. 70.8mJ vs. 74.1mJ. At the 20M scale, **E-SpikeFormer**

vs. DeiT-S [72] vs. PVT-S [74]: Param, **19M** vs. 22M vs. 24.5M; Acc, **79.8%** vs. 79.8% vs. 79.8%; Power, **5.9 mJ** vs. 21.2mJ vs. 35.0mJ. This is the first time that the SNN domain can compete with the performance of classic ANN-ViT models.

2) *Object Detection:* Before the emergence of Meta-SpikeFormer [28], the SNN domain could only process object detection tasks through specific architectural designs, and the accuracy was low. Meta-SpikeFormer enable SNN to process dense prediction tasks such as object detection and semantic segmentation in a unified way. In this work, we test E-SpikeFormer on the COCO dataset [33], which includes 118K training images (train2017) and 5K validation images (val2017). Results are shown in Table II, E-SpikeFormer achieves SOTA results in SNNs, and exceeds or is comparable to classical models in ANN but with a lower energy cost. For example, **E-SpikeFormer** vs. Meta-SpikeFormer [28] vs. PVT-Tiny [74]: Param, **38.7M** vs. 75.0M vs. 32.9M; mAP@0.5, **58.8%** vs. 51.2% vs. 56.7%; Power, **119.5mJ** vs.

TABLE II: Performance of detection on COCO val2017 [33]. * These methods exploit the pre-trained models on ImageNet as the backbone, then add detection heads for fine-tuning.

Methods	Architecture	Param (M)	Power (mJ)	Step	mAP @50(%)
ANN	ResNet-18 [50]	31.2	890.6	1	54.0
	PVT-Tiny [74]	32.9	1040.5	1	56.7
	DETR-DC5 [90]	41.0	860.2	1	55.7
	Spiking-Yolo [91]	10.2	-	3500	25.7
ANN2SNN	Spike Calibration [92]	17.1	-	512	45.3
	Fast-SNN [12]	25.1	-	15	46.4
	Spiking Retina [93]	11.3	-	4 × 1	28.5
Direct training	EMS-Res-SNN [94]	26.9	-	4 × 1	50.1
	Meta-SpikeFormer * [28]	34.9	49.5	1 × 1	44.0
		75.0	140.8	1 × 1	51.2
		38.7	56.2	1 × 2	41.8
Direct training	E-SpikeFormer * (This Work)	38.7	94.5	1 × 4	58.4
		38.7	119.5	1 × 8	58.8

TABLE III: Performance of segmentation on ADE20K [34]. * These methods use the pre-trained models on ImageNet as the backbone, then add segmentation heads for fine-tuning.

Methods	Architecture	Param (M)	Power (mJ)	Step	MIoU (%)
ANN	ResNet-18 [50]	15.5	147.1	1	32.9
	PVT-Tiny [74]	17.0	152.7	1	35.7
	PVT-Small [74]	28.2	204.7	1	39.8
	InternImage-T [95]	59.0	2171.2	1	48.1
	ConvNeXt-T [80]	60.0	2173.5	1	45.8
	DeepLab-V3 [96]	68.1	1240.6	1	42.7
		16.5	22.1	1 × 1	32.3
Direct training	Meta-SpikeFormer * [28]	16.5	88.1	4 × 1	33.6
		59.8	46.6	1 × 1	34.8
		59.8	183.6	4 × 1	35.3
Direct training	E-SpikeFormer * (This Work)	11.0	18.4	1 × 2	31.9
		11.0	27.2	1 × 4	40.1
		11.0	33.6	1 × 8	41.4

140.8mJ vs. 1040.5mJ.

3) *Semantic Segmentation*: ADE20K [34] is a challenging semantic segmentation benchmark, including 20K and 2K images in the training and validation set, respectively, and covering 150 categories. Meta-SpikeFormer [28] reported the first results of SNN on the ADE20K dataset. We follow the experimental setting in Meta-SpikeFormer to test our E-SpikeFormer on ADE20K. Our model performs well on ADE20K. For example, **E-SpikeFormer** vs. Meta-SpikeFormer [28]: Param, **11.0M** vs. 59.8M ; MIoU, **41.4%** vs. 35.3%; Power, **33.6mJ** vs. 183.6mJ.

4) *Nuromorphic Human Action Recognition*: HAR-DVS [35] is the largest event-based Human Activity Recognition (HAR) dataset currently, containing 300 classes and 107,646 samples, acquired by a DAVIS346 camera with a spatial resolution of 346×260 . We use HAR-DVS to evaluate E-SpikeFormer, data preprocessing and experimental design (please see Section S2.2) follow the previous Meta-SpikeFormer. We handle HAR-DVS in a direct training manner with $T = 8$. Meta-SpikeFormer achieves comparable accuracy to ANNs and is better than prior SNNs (Table IV). We also verify the effectiveness of our methods on neuromorphic

TABLE IV: Performance of neuromorphic human action recognition on HAR-DVS [35].

Methods	Architecture	Param (M)	Power (mJ)	Step	Acc.(%)
ANN	SlowFast [97]	33.6	-	8 × 1	46.5
	ACTION-Net [98]	27.9	-	8 × 1	46.9
	TimeSformer [99]	121.2	-	8 × 1	50.8
Direct training	Res-SNN-34 [17]	21.8	-	8 × 1	46.1
	ASA-SNN [100]	41.5	-	8 × 1	47.1
	Meta-SpikeFormer [28]	18.3	8.0	8 × 1	47.5
Direct training	E-SpikeFormer	18.7	18.1	8 × 4	48.9
	(This Work)	18.7	23.5	8 × 8	49.2

TABLE V: Ablation studies of SFA and architecture design.

Methods	Param (M)	Step	Power (mJ)	Acc.(%)
Baseline [28]	10.0	1 × 1	3.3	68.5
	10.0	4 × 1	11.9	71.3
+ Our SFA training	10.0	1 × 4	7.3 (-4.6)	77.8 (+6.5)
+ Our architecture	10.0	1 × 4	3.0 (-4.3)	78.5 (+0.7)

CIFAR10-DVS, and the experimental results are given in the supplementary materials (Section S4).

B. Ablation Study

In Table V, we conduct rigorous ablation experiments to verify the gains of SFA training and E-SpikeFormer compared to the baseline (Meta-SpikeFormer [28]) on ImageNet.

i) Baseline vs. **Baseline+SFA**: Step, 4×1 vs. 1×4 ; Power, 11.9mJ vs. **7.3mJ(-4.6mJ)**; Acc, 71.3% vs. **77.8%(+6.5%)**. This implies that the strategy of SFA is far more effective and efficient than existing directly training SNN.

ii) We advance the SNN architecture, Baseline+SFA vs. **E-SpikeFormer+SFA**: Power, 7.3mJ vs. **3.7mJ(-4.3mJ)**; Acc, 77.8% vs. **78.5%(+0.7%)**. The proposed efficient architecture design further improves accuracy while reducing power.

In addition, we observe the effect of SFA method on energy cost, and conduct in depth analysis in Section V-C. Together, by combining SFA and E-SpikeFormer, compared with the 10M baseline [28], we simultaneously achieve a **7.2%** performance improvement, **5.3×** training time acceleration (12 hours vs. 63 hours on 8 NVIDIA-A100-40G with 200 epoch), and **3.7×** inference energy efficiency.

Due to space limitations, ablation experiments on MIM training are placed in Section S5 of the supplementary materials, including MIM mask ratio, decoder width and depth, and studies on MIM and distillation.

C. Advantages of SFA in Power and Performance

Compared with ANN2SNN and direct training, the SFA method implies a novel spike firing pattern (Fig. 3). We here focus on the following question: “How does the SFA method optimize the power and performance of SNNs?”

We primarily examine the spike distribution under different spike firing patterns. First, we define the *Network Spike Firing Rate (NSFR)*: at timestep t , SNN’s NSFR is the ratio of spikes

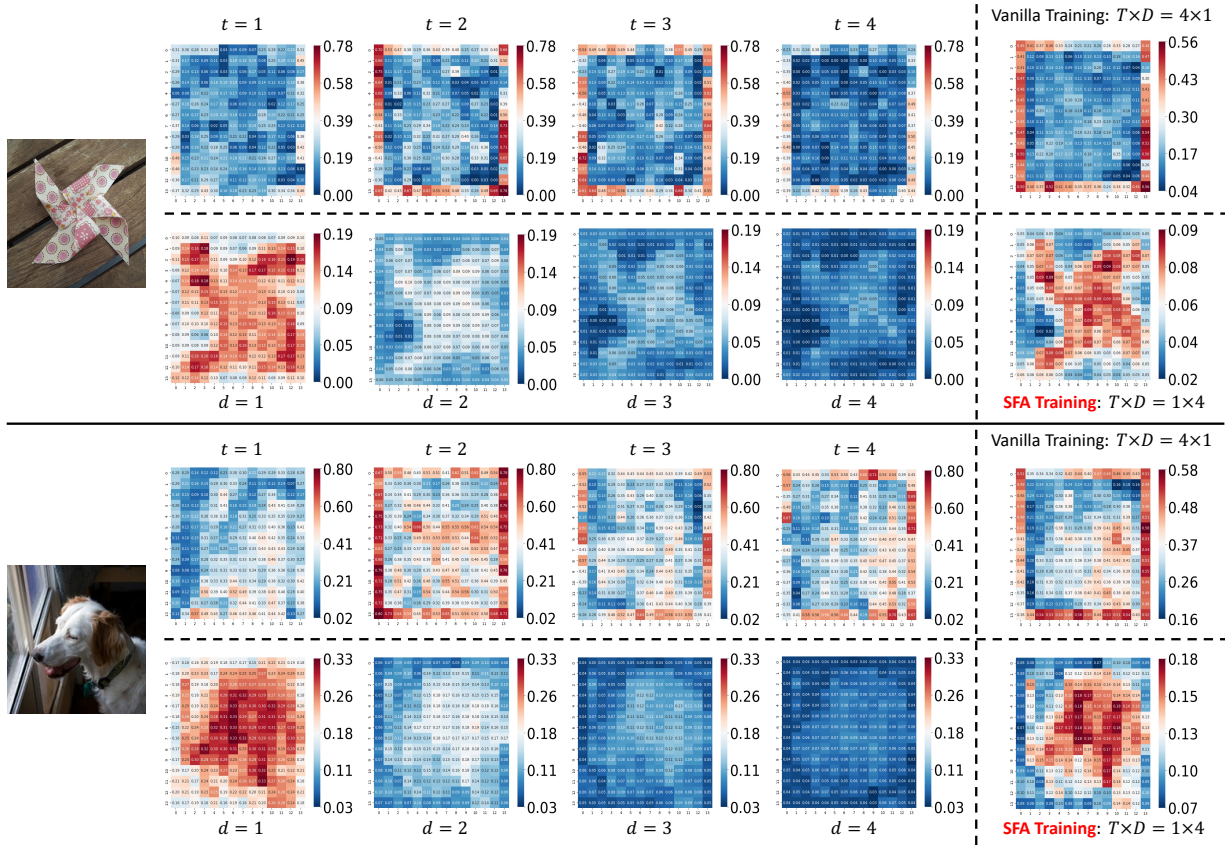


Fig. 8: Visualization of spike distribution in vanilla and SFA training on ImageNet. From top to bottom: vanilla and SFA training. For a single sample, the output spike maps of a certain SNN layer in the form of the 4D $([T, C, H, W])$. We first averaged the 4D spike maps into a 3D map $([T, H, W])$ over the channel dimension. Since $T = 4$, we can split the 3D map into four 2D maps (the four maps in the middle of this figure). Then, we averaged the 4D spike maps into a 2D map $([H, W])$ over the temporal and channel dimension, resulting in the two rightmost spike maps. For a single map, each pixel represents the firing rate of a spiking neuron. The redder the pixel, the higher the spike firing rate; the bluer the pixel, the closer the spike firing rate is to 0. The four spike maps in the middle show the differences in the temporal dimension between the spike distributions obtained by vanilla and SFA training: spike firing rate does not change much from left to right in vanilla training (first row); spike firing rate decreases from left to right in SFA training (second row). The two spike maps on the far right show that the spike features obtained by SFA training are clearer, and the firing rate is lower.

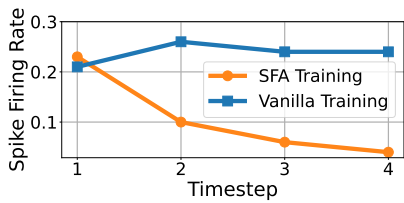


Fig. 9: Network Spike Firing Rate (NSFR) of vanilla ($T \times D = 4 \times 1$) training and SFA ($T \times D = 1 \times 4$) training at each timestep. Experimental settings are the same as the ablation studies in Table V. The sum of NSFR over all timesteps implies the average spike firing profile over the entire dataset: in vanilla training and SFA training, each spiking neuron fires an average of 0.96 and 0.39 spikes, respectively.

produced over all the neurons to the total number of neurons in this timestep. NSFR is used to evaluate the change of the same network's spike distribution at different timesteps. By

default, we count spike firing rates of the samples in the test set and get their mean as the final data. Second, we *visualizes the spike maps* of the last layer of the last block of the 10M model in Fig. 8. The output spike maps of a certain SNN layer in the form of the 4D $([T, C, H, W])$, which can be averaged into a 3D map $([T, H, W])$ over the channel dimension. Since $T = 4$, we can split the 3D map into four 2D maps, which are the four maps in the middle of Fig. 8. Each pixel of spike map represents the firing rate of a spiking neuron. The redder the pixel, the higher spike firing rate; the bluer the pixel, the closer the spike firing rate is to zero. Then we have the following observations:

i) **Power optimization.** Fig. 9 shows the NSFRs of vanilla and SFA training SNNs. In vanilla training, NSFR at each timestep is close (the blue line). In contrast, the NSFR of the SNN trained by SFA method will decrease as the timestep increases (the orange line). Fig. 8 shows the spike distributions for the two samples obtained using the vanilla and SFA

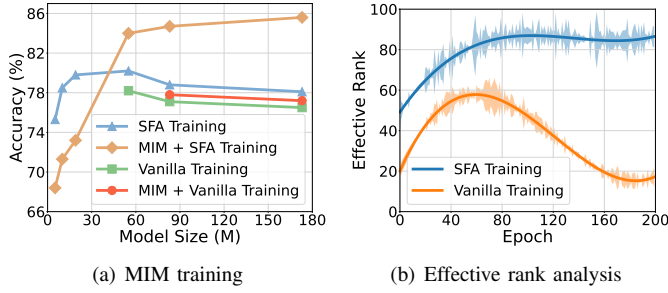


Fig. 10: Binary spike firing interferes with the scaling of SNNs. (a) Ablation experiments on MIM pre-training and SFA training. In direct training with binary spike firing, performance degradation still occurs exploiting MIM pre-training. By contrast, in SFA training using integer activations, MIM pretraining effectively scales up SNNs. (b) Experimental results on effective rank [101] in MIM-SNN.

training. It can be seen that the color of the spike map obtained by SFA training (second row of each sample) changes from red to blue from left to right, which means that as the timestep increases, the spike firing rate decreases. In contrast, the spike map obtained by vanilla training (first row of each) does not change much in color. This phenomenon coincides with the data in Fig. 9. And, the highest spike firing rates of spiking neurons (the value of the reddest pixel in the figure) obtained by SFA and vanilla training are 0.19 and 0.78, respectively.

ii) **Performance optimization.** In Fig. 8, the 4D spike maps can also be averaged into a 2D map ($[H, W]$) over the temporal and channel dimension, resulting in the two rightmost spike maps. It can be observed that compared with vanilla training, the spike feature obtained by SFA training are clearer and the firing rate is lower.

D. Binary Spike Firing Mechanism is NOT Conducive to the Scaling of SNNs

We analyze why directly scaling E-SpikeFormer results in performance degradation from the perspective of encoder design in Section III-D. The issue arises due to vanilla spike convolution leaking information after the mask operation in MIM is superimposed. Spike sparse convolution is proposed to address this. Moreover, we observe that binary spike firing also interferes with the scaling of SNNs. As shown in Fig. 10(a), we analyze the scaling of SNNs from two orthogonal perspectives: with and without MIM-SNN pre-training and SFA versus direct training. As shown by the orange line, the MIM strategy can effectively help SNNs scaling when trained using the SFA method. However, in the vanilla training with binary spikes, performance degradation still occurs after using the MIM strategy (red line).

We argue that the potential reason is that the binary spikes affect the feature extraction ability of the encoder part in MIM-SNN. Specifically, the feature extraction capability can be measured by the effective rank [73], [101]:

$$\text{Rank}(\mathbf{A}) = \exp\left(-\sum_{k=1}^n p_k \log(p_k)\right), \quad (29)$$

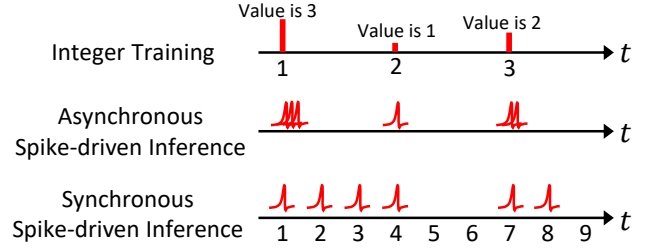


Fig. 11: Possible implementations of spike-driven inference on neuromorphic chips. In the figure, it is assumed that $T = D = 3$. Asynchronous spike-driven has no global clock [9], and the integer D is converted into D spikes fired in a very short time window during inference. Synchronous spike-driven expands the T timesteps in integer training to $T \times D$ timesteps during inference.

where $p_k = \frac{\sigma_k}{\|\sigma\|_1}$ is the distribution of singular values. $\sigma_1, \dots, \sigma_n$ is the singular value on the diagonal matrix Σ . It can be obtained by the SNN encoder's output $\mathbf{Z}$, where $\mathbf{Z} = \mathbf{U}\Sigma\mathbf{H}^T$. A high effective rank indicates that rich feature information is being extracted, while a low effective rank suggests the opposite.

Experimental results are given in Fig. 10(b). In MIM-SNN with vanilla training, the effective rank decreases over training (Fig. 10(b)), suggesting collapsing feature representation into a lower-dimensional space, potentially harming generalization and task performance. This could lead to difficulty distinguishing input samples, a lack of complexity capture, and performance degradation. By contrast, in MIM-SNN with SFA, the effective rank increases and tends to be stable during training, indicating robust representations and better fine-tuning performance.

VI. DISCUSSION AND SUMMARY

This section discusses the way in which the SFA method and MIM-based pre-trained SNNs can be executed on neuromorphic chips. And, we summarize the profound effects of changes in spike firing patterns induced by the SFA method.

A. Implementation of SFA on Neuromorphic Chip

The spike firing patterns for vanilla and SFA training are different (Fig. 3). In ANN2SNN and vanilla direct training, the spikes at multi-timesteps can be considered random. By contrast, the spikes in SFA training are concentrated and continuous. This is crucial and implies that the mechanisms of implementation of these two methods on neuromorphic chips are different.

Fig. 11 shows in what way the proposed SFA method can be executed on a neuromorphic chip:

i) **Synchronous implementation.** In this case, integers are synchronously converted to spikes. Specifically, since synchronous neuromorphic chips contain a global clock, T timesteps are re-expanded into $T \times D$ timesteps. In static visual tasks, only the timesteps during inference have changed. In dynamic visual tasks, different spiking neurons are required

at different timesteps. Tianjic theoretically supports these operations [6].

ii) **Asynchronous implementation.** In this case, the implementation of “a single timestep is expanded D times” on the chip is “the asynchronous neuromorphic chip emits a series of spikes (up to D) in a very short time window.” Asynchronous neuromorphic chips do not have a global clock, so there is actually no concept of timestep (please refer to the “Synchronous Training and Asynchronous Deployment” part in [9] for more details), and the spike firing between different spiking neurons is independent. When there is an event input, a part of spiking neurons on the chip will be activated, and the rest of the time they are in a idle state. And, there is no need to switch the working mode of the spiking neuron in order to extend the additional timesteps. Speck [9] supports this operation.

We make the following discussion about the pros and cons of implementing SFA methods synchronously or asynchronously:

i) The SFA method is implemented by either synchronous or asynchronous neuromorphic chips. However, vanilla training SNNs are difficult to implement with asynchronous chips. As shown in Fig. 3, vanilla training SNNs must execute all timesteps to reach the spike firing approximation, thus requiring a globally synchronized clock. By contrast, SFA training SNNs only need to complete the fire of a consecutive string of D spikes, and the spiking neurons can be rested. No global clock is required.

ii) When D is small, as well as the spike firing rate is small, the asynchronous spike-driven implementation of SFA method certainly has outstanding advantages in terms of power and latency. The risk point is that when D is not limited, the communication bandwidth of the chip may be exceeded in the condition of a large number of spikes being fired in a short period of time. But this risk is controllable.

Therefore, considering the unique characteristics of the SFA method, it is evident that it is more compatible with asynchronous implementation, which has significant implications for the design and development of algorithm-driven neuromorphic chips. Another interesting and noteworthy point is that for the SFA method, an increase in D may lead to higher energy consumption, but the extent of this increase is closely related to the specific task. For instance, adjusting 1×4 to 1×8 results in a significantly higher increase in energy cost in image classification (Table I) than in detection (Table II) and segmentation (Table III) tasks.

B. Neuromorphic Chips Naturally Support MIM-SNN Pre-training

Spike convolution is performed on neuromorphic chips in a spike-driven manner. Event-by-event sparse processing enables neuromorphic computing to enjoy the advantages of low latency and low power consumption. The system state is changed based on each spike input. Upon receiving a spike, the SNN core identifies the corresponding weight and target neuron position via address search and updates the target neuron state accordingly [9].

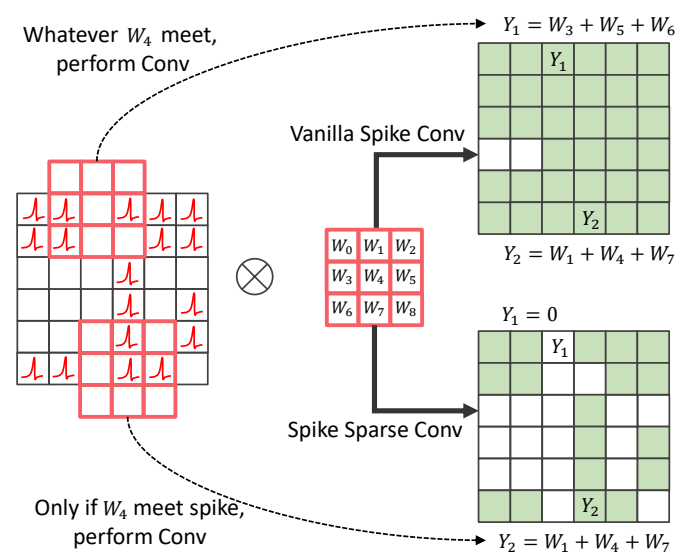


Fig. 12: Comparison of Spike Sparse Conv (SSC) and Vanilla Spike Conv (VSC). On a neuromorphic chip, when a spike occurs, the address mapping function finds the synapses and neurons that need to be added, and then takes out the corresponding weights to perform the addition operations. The only difference between VSC and SSC is the addressing mapping function. In SSC, it is specified that the convolution is performed only if there is a spike input at the position corresponding to W_4 (the center position of the convolution kernel). VSC does not have this restriction.

Spike Sparse Convolution (SSC) is proposed to solve the information leakage problem during MIM-SNN pre-training. Since the Vanilla Spike Convolution (VSC) itself is a sparse computation convolution, we are interested in the difference and connection between VSC and SSC. As shown in Fig. 12, we observe that SSC actually only has one more judgment condition than VSC when performing spike convolution. In SSC, the convolution is performed only if there is a spike input at the position corresponding to W_4 (the center position of the convolution kernel). Therefore, SSC can always pass the information in the spike feature map to the corresponding position of the next layer. This is important in cases where a MIM encoder is required, as most of the information needs to be masked. Moreover, masked images are first divided into patches, and SSC records the positions of these masks in E-SpikeFormer’s MIM pretraining. Then SSC operates exclusively at unmasked locations where the central kernel firing spikes.

Given the commonality of VSC and SSC, we only need to slightly modify the addressing mapping function corresponding to VSC to execute SSC on the neuromorphic chip. In other words, current neuromorphic chips can support on-chip learning based on MIM pre-training.

C. Summary

After discussing the hardware implementation, we summarize here the effect of spike firing patterns on SNNs. Various

spike firing patterns, e.g., $\{\mathbf{S}^l[d]\}_D$ in ANN2SNN and vanilla training versus $\{\hat{\mathbf{S}}^l[d]\}_D$ in our SFA training, result in varying spike distributions. These differences subsequently influence the training, power, performance, scaling, and hardware implementation of SNNs. All three training methods can be thought of as rate coding. We still use Fig. 3 as an example of the difference between them. ANN2SNN and direct training must run all ten timesteps to complete the rate coding. SFA training, on the other hand, only needs to fire spikes at the first few timesteps. This minor yet significant design improvement has profound impacts:

i) **Reducing training costs (Fig. 6(b) and Fig. 7).** When dealing with static vision tasks, SFA requires only one-timestep training, avoiding the huge training memory and time cost with traditional multi-timestep direct training.

ii) **Less power cost (Fig. 8 and Fig. 9).** In the static task, the inference stage of the SFA method only uses the image input once, without the need to repeat the input inference as in the traditional direct training. On the other hand, the spike distribution obtained by the SFA method has an spike firing rate that decreases progressively with timesteps.

iii) **Higher performance (Fig. 8).** The SFA method yields a clearer spike map.

iv) **Easy to scale (Fig. 10(b)).** Integer training is more likely to learn good features.

v) **More suitable for execution on asynchronous neuromorphic chips (Fig. 11).** The inference process in the SFA method is naturally suited to be deployed on asynchronous neuromorphic chips, which can fully embody the low-power and low-latency features of SNNs.

VII. CONCLUSION

We address the performance and training consumption gap between SNNs and ANNs. A key contribution is identifying the mechanistic flaw of binary spike firing in spiking neurons. To overcome these limitations, we propose a Spike Firing Approximation (SFA) method. This method is based on integer training and spike-driven inference, aiming to optimize the spike firing pattern of spiking neurons. Our results demonstrate that optimization the spike firing pattern leads to comprehensive improvements in SNNs, including enhanced training efficiency, reduced power consumption, improved performance, easier scalability, and better utilization of neuromorphic chips. Additionally, we develop an efficient spike-driven Transformer architecture and a spike masked autoencoder to prevent performance degradation during SNN scaling. By addressing the training and performance challenges of large-scale SNNs, we pave the way for a new era in neuromorphic computing.

ACKNOWLEDGMENT

This work was partially supported by CAS Project for Young Scientists in Basic Research (YSBR-116), National Distinguished Young Scholars (62325603), National Natural Science Foundation of China (62236009, U22A20103, 62441606, 62406322), Beijing Science and Technology Plan (Z241100004224011), Beijing Natural Science Foundation for Distinguished Young Scholars (JQ21015), China Postdoctoral Science Foundation (GZB20240824, 2024M753497).

REFERENCES

- [1] K. Roy, A. Jaiswal, and P. Panda, "Towards spike-based machine intelligence with neuromorphic computing," *Nature*, vol. 575, no. 7784, pp. 607–617, 2019.
- [2] C. D. Schuman, S. R. Kulkarni, M. Parsa, J. P. Mitchell, B. Kay *et al.*, "Opportunities for neuromorphic computing algorithms and applications," *Nature Computational Science*, vol. 2, no. 1, pp. 10–19, 2022.
- [3] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.
- [4] P. A. Merolla, J. V. Arthur, R. Alvarez-Icaza, A. S. Cassidy, J. Sawada, F. Akopyan, B. L. Jackson, N. Imam, C. Guo, Y. Nakamura *et al.*, "A million spiking-neuron integrated circuit with a scalable communication network and interface," *Science*, vol. 345, no. 6197, pp. 668–673, 2014.
- [5] M. Davies, N. Srinivasa, T.-H. Lin, G. Chinya, Y. Cao, S. H. Choday, G. Dimou, P. Joshi, N. Imam, S. Jain *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.
- [6] J. Pei, L. Deng *et al.*, "Towards artificial general intelligence with hybrid tianjic chip architecture," *Nature*, vol. 572, no. 7767, pp. 106–111, 2019.
- [7] B. Yin, F. Corradi, and S. M. Bohté, "Accurate and efficient time-domain classification with adaptive spiking recurrent neural networks," *Nature Machine Intelligence*, vol. 3, no. 10, pp. 905–913, 2021.
- [8] A. Rao, P. Plank, A. Wild, and W. Maass, "A long short-term memory for ai applications in spike-based neuromorphic hardware," *Nature Machine Intelligence*, vol. 4, no. 5, pp. 467–479, 2022.
- [9] M. Yao, O. Richter, G. Zhao, N. Qiao, Y. Xing, D. Wang, T. Hu, W. Fang, T. Demirci, M. De Marchi, L. Deng, T. Yan, C. Nielsen, S. Sheik, C. Wu, Y. Tian, B. Xu, and G. Li, "Spike-based dynamic computing with asynchronous sensing-computing neuromorphic chip," *Nature Communications*, vol. 15, no. 1, p. 4464, 2024.
- [10] Y. Hu, H. Tang, and G. Pan, "Spiking deep residual networks," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 8, pp. 5200–5205, 2023.
- [11] N. Rathi, G. Srinivasan, P. Panda, and K. Roy, "Enabling deep spiking neural networks with hybrid conversion and spike timing dependent backpropagation," in *International Conference on Learning Representations*, 2020.
- [12] Y. Hu, Q. Zheng, X. Jiang, and G. Pan, "Fast-snn: Fast spiking neural network by converting quantized ann," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 14546–14562, 2023.
- [13] J. Wu, C. Xu, X. Han, D. Zhou, M. Zhang, H. Li, and K. C. Tan, "Progressive tandem learning for pattern recognition with deep spiking neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 11, pp. 7824–7840, 2021.
- [14] Z. Wang, Y. Fang, J. Cao, Q. Zhang, Z. Wang, and R. Xu, "Masked spiking transformer," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 1761–1771.
- [15] Y. Wu, L. Deng, G. Li, J. Zhu, and L. Shi, "Spatio-temporal back-propagation for training high-performance spiking neural networks," *Frontiers in Neuroscience*, vol. 12, p. 331, 2018.
- [16] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.
- [17] W. Fang, Z. Yu, Y. Chen, T. Huang, T. Masquelier, and Y. Tian, "Deep residual learning in spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 34, pp. 21 056–21 069, 2021.
- [18] H. Zheng, Y. Wu, L. Deng, Y. Hu, and G. Li, "Going deeper with directly-trained larger spiking neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, 2021, pp. 11 062–11 070.
- [19] Y. Hu, L. Deng, Y. Wu, M. Yao, and G. Li, "Advancing spiking neural networks toward deep residual learning," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–15, 2024.
- [20] M. Yao, G. Zhao, H. Zhang, Y. Hu, L. Deng, Y. Tian, B. Xu, and G. Li, "Attention spiking neural networks," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 9393–9410, 2023.
- [21] X. Qiu, R.-J. Zhu, Y. Chou, Z. Wang, L.-j. Deng, and G. Li, "Gated attention coding for training high-performance and efficient spiking neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 1, 2024, pp. 601–610.

- [22] X. Luo, M. Yao, Y. Chou, B. Xu, and G. Li, "Integer-valued training and spike-driven inference spiking neural network for high-performance and energy-efficient object detection," *arXiv preprint arXiv:2407.20708*, 2024.
- [23] M. Yao, H. Gao, G. Zhao, D. Wang, Y. Lin, Z. Yang, and G. Li, "Temporal-wise attention spiking neural networks for event streams classification," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10 221–10 230.
- [24] M. Yao, H. Zhang, G. Zhao, X. Zhang, D. Wang, G. Cao, and G. Li, "Sparsen spiking activity can be better: Feature refine-and-mask spiking neural network for event-based visual recognition," *Neural Networks*, vol. 166, pp. 410–423, 2023.
- [25] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li, and Y. Tian, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023.
- [26] Y. Guo, Y. Chen, L. Zhang, X. Liu, Y. Wang, X. Huang, and Z. Ma, "Im-loss: information maximization loss for spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 156–166, 2022.
- [27] L. Deng, Y. Wu, X. Hu, L. Liang, Y. Ding, G. Li, G. Zhao, P. Li, and Y. Xie, "Rethinking the performance comparison between snns and anns," *Neural Networks*, vol. 121, pp. 294–307, 2020.
- [28] M. Yao, J. Hu, T. Hu, Y. Xu, Z. Zhou, Y. Tian, B. XU, and G. Li, "Spike-driven transformer v2: Meta spiking neural network architecture inspiring the design of next-generation neuromorphic chips," in *The Twelfth International Conference on Learning Representations*, 2024.
- [29] M. Yao, J. Hu, Z. Zhou, L. Yuan, Y. Tian, B. Xu, and G. Li, "Spike-driven transformer," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 64 043–64 058.
- [30] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 16 000–16 009.
- [31] K. Tian, Y. Jiang, C. Lin, L. Wang, Z. Yuan *et al.*, "Designing bert for convolutional networks: Sparse and hierarchical masked modeling," in *The Eleventh International Conference on Learning Representations*, 2022.
- [32] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2009, pp. 248–255.
- [33] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, "Microsoft coco: Common objects in context," in *European Conference on Computer Vision*. Springer, 2014, pp. 740–755.
- [34] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, "Scene parsing through ade20k dataset," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 633–641.
- [35] X. Wang, Z. Wu, B. Jiang, Z. Bao, L. Zhu, G. Li, Y. Wang, and Y. Tian, "Hardvs: Revisiting human activity recognition with dynamic vision sensors," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 6, 2024, pp. 5615–5623.
- [36] M. Xiao, Q. Meng, Z. Zhang, D. He, and Z. Lin, "Online training through time for spiking neural networks," *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 717–20 730, 2022.
- [37] Q. Meng, M. Xiao, S. Yan, Y. Wang, Z. Lin, and Z.-Q. Luo, "Towards memory-and time-efficient backpropagation for training spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 6166–6176.
- [38] Y. Lin, Y. Hu, S. Ma, D. Yu, and G. Li, "Rethinking pretraining as a bridge from anns to snns," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–14, 2022.
- [39] H. Zhang and Y. Zhang, "Memory-efficient reversible spiking neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 15, 2024, pp. 16 759–16 767.
- [40] J. Hu, M. Yao, X. Qiu, Y. Chou, Y. Cai, N. Qiao, Y. Tian, B. Xu, and G. Li, "High-performance temporal reversible spiking neural networks with $\mathcal{O}(l)$ training memory and $\mathcal{O}(1)$ inference cost," in *Proceedings of the 41st International Conference on Machine Learning*, vol. 235, 2024, pp. 19 516–19 530.
- [41] S. S. Chowdhury, N. Rathi, and K. Roy, "Towards ultra low latency spiking neural networks for vision and sequential tasks using temporal pruning," in *European Conference on Computer Vision*. Springer, 2022, pp. 709–726.
- [42] Y. Li, T. Geller, Y. Kim, and P. Panda, "Seenn: Towards temporal spiking early exit neural networks," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 63 327–63 342.
- [43] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.
- [44] K. He, X. Zhang, and S. Ren, "Identity mappings in deep residual networks," in *European Conference on Computer Vision*. Springer, 2016, pp. 630–650.
- [45] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, vol. 30, 2017.
- [46] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2020.
- [47] J. Zhang, L. Tang, Z. Yu, J. Lu, and T. Huang, "Spike transformer: Monocular depth estimation for spiking camera," in *European Conference on Computer Vision*. Springer, 2022, pp. 34–52.
- [48] Q. Wang, T. Zhang, M. Han, Y. Wang, D. Zhang, and B. Xu, "Complex dynamic neurons improved spiking transformer network for efficient automatic speech recognition," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 1, 2023, pp. 102–109.
- [49] Z. Zhou, Y. Zhu, C. He, Y. Wang, S. YAN, Y. Tian, and L. Yuan, "Spikformer: When spiking neural network meets transformer," in *The Eleventh International Conference on Learning Representations*, 2023.
- [50] W. Yu, M. Luo, P. Zhou, C. Si, Y. Zhou, X. Wang, J. Feng, and S. Yan, "Metaformer is actually what you need for vision," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 10 819–10 829.
- [51] Y. Chou, M. Yao, K. Wang, Y. Pan, R.-J. Zhu, J. Wu, Y. Zhong, Y. Qiao, B. XU, and G. Li, "MetaLA: Unified optimal linear approximation to softmax attention map," in *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [52] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [53] H. Bao, L. Dong, S. Piao, and F. Wei, "Beit: Bert pre-training of image transformers," *arXiv preprint arXiv:2106.08254*, 2021.
- [54] J. Zhou, C. Wei, H. Wang, W. Shen, C. Xie, A. Yuille, and T. Kong, "Image bert pre-training with online tokenizer," in *International Conference on Learning Representations*, 2021.
- [55] P. Gao, T. Ma, H. Li, Z. Lin, J. Dai, and Y. Qiao, "Mcmae: Masked convolution meets masked autoencoders," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 35 632–35 644.
- [56] S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "Convnext v2: Co-designing and scaling convnets with masked autoencoders," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, June 2023, pp. 16 133–16 142.
- [57] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks: Bringing the power of gradient-based optimization to spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.
- [58] W. Gerstner, W. M. Kistler, R. Naud, and L. Paninski, *Neuronal dynamics: From single neurons to networks and models of cognition*. Cambridge University Press, 2014.
- [59] G. Indiveri, B. Linares-Barranco, T. J. Hamilton, A. v. Schaik, R. Etienne-Cummings, T. Delbruck, S.-C. Liu, P. Dudek, P. Häfliger, S. Renaud *et al.*, "Neuromorphic silicon neuron circuits," *Frontiers in Neuroscience*, vol. 5, p. 73, 2011.
- [60] W. Fang, Z. Yu, Z. Zhou, D. Chen, Y. Chen, Z. Ma, T. Masquelier, and Y. Tian, "Parallel spiking neurons with high efficiency and ability to learn long-term dependencies," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 53 674–53 687.
- [61] W. Ponghiran and K. Roy, "Spiking neural networks with improved inherent recurrence dynamics for sequential learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 7, 2022, pp. 8001–8008.
- [62] G. Shen, D. Zhao, T. Li, J. Li, and Y. Zeng, "Are conventional snns really efficient? a perspective from network quantization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024, pp. 27 538–27 547.

- [63] Z. Wu, H. Zhang, Y. Lin, G. Li, M. Wang, and Y. Tang, "Liaf-net: Leaky integrate and analog fire network for lightweight and efficient spatiotemporal information processing," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 11, pp. 6249–6262, 2021.
- [64] Y. Wu, L. Deng, G. Li, J. Zhu, Y. Xie, and L. Shi, "Direct training for spiking neural networks: Faster, larger, better," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 1311–1318.
- [65] N. Rathi and K. Roy, "Diet-snn: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 34, no. 6, pp. 3174–3182, 2023.
- [66] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.
- [67] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *International Conference on Machine Learning*. PMLR, 2015, pp. 448–456.
- [68] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 1251–1258.
- [69] H. Cai, C. Gan, and S. Han, "Efficientvit: Enhanced linear attention for high-resolution low-computation visual recognition," *arXiv preprint arXiv:2205.14756*, 2022.
- [70] S. N. Wadkar and A. Chaurasia, "Mobilevitv3: Mobile-friendly vision transformer with simple and effective fusion of local, global and input features," *arXiv preprint arXiv:2209.15159*, 2022.
- [71] J. Zhang, X. Li, J. Li, L. Liu, Z. Xue, B. Zhang, Z. Jiang, T. Huang, Y. Wang, and C. Wang, "Rethinking mobile block for efficient attention-based models," in *Proceedings of IEEE/CVF International Conference on Computer Vision*, 2023, pp. 1389–1400.
- [72] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *International Conference on Machine Learning*. PMLR, 2021, pp. 10347–10357.
- [73] Q. Zhang, Y. Wang, and Y. Wang, "How mask matters: Towards theoretical understandings of masked autoencoders," *Advances in Neural Information Processing Systems*, vol. 35, pp. 27 127–27 139, 2022.
- [74] W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao, "Pyramid vision transformer: A versatile backbone for dense prediction without convolutions," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 568–578.
- [75] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 10012–10022.
- [76] W. Yu, C. Si, P. Zhou, M. Luo, Y. Zhou, J. Feng, S. Yan, and X. Wang, "Metaformer baselines for vision," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [77] L. Yuan, Y. Chen, T. Wang, W. Yu, Y. Shi, Z.-H. Jiang, F. E. Tay, J. Feng, and S. Yan, "Tokens-to-token vit: Training vision transformers from scratch on imagenet," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 558–567.
- [78] X. Ding, X. Zhang, J. Han, and G. Ding, "Scaling up your kernels to 31x31: Revisiting large kernel design in cnns," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 963–11 975.
- [79] J. Yang, C. Li, P. Zhang, X. Dai, B. Xiao, L. Yuan, and J. Gao, "Focal attention for long-range interactions in vision transformers," *Advances in Neural Information Processing Systems*, vol. 34, pp. 30 008–30 022, 2021.
- [80] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A convnet for the 2020s," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 976–11 986.
- [81] A. Srinivas, T.-Y. Lin, N. Parmar, J. Shlens, P. Abbeel, and A. Vaswani, "Bottleneck transformers for visual recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 16 519–16 529.
- [82] W. Wang, W. Chen, Q. Qiu, L. Chen, B. Wu, B. Lin, X. He, and W. Liu, "Crossformer++: A versatile vision transformer hinging on cross-scale attention," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 5, pp. 3123–3136, 2024.
- [83] T. Bu, W. Fang, J. Ding, P. Dai, Z. Yu, and T. Huang, "Optimal ann-snn conversion for high-accuracy and ultra-low-latency spiking neural networks," in *International Conference on Learning Representations*, 2021.
- [84] Y. Hu, Q. Zheng, X. Jiang, and G. Pan, "Fast-snn: Fast spiking neural network by converting quantized ann," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 12, pp. 14 546–14 562, 2023.
- [85] Z. Wang, Y. Zhang, S. Lian, X. Cui, R. Yan, and H. Tang, "Toward high-accuracy and low-latency spiking neural networks with two-stage optimization," *IEEE Transactions on Neural Networks and Learning Systems*, pp. 1–15, 2023.
- [86] S. Deng, Y. Li, S. Zhang, and S. Gu, "Temporal efficient training of spiking neural network via gradient re-weighting," in *International Conference on Learning Representations*, 2022.
- [87] Q. Meng, M. Xiao, S. Yan, Y. Wang, Z. Lin, and Z.-Q. Luo, "Training high-performance low-latency spiking neural networks by differentiation on spike representation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 12 444–12 453.
- [88] Z. Zhou, K. Che, W. Fang, K. Tian, Y. Zhu, S. Yan, Y. Tian, and L. Yuan, "Spikformer v2: Join the high accuracy club on imagenet with an snn ticket," *arXiv preprint arXiv:2401.02020*, 2024.
- [89] H. Li, H. Liu, X. Ji, G. Li, and L. Shi, "Cifar10-dvs: an event-stream dataset for object classification," *Frontiers in Neuroscience*, vol. 11, p. 309, 2017.
- [90] X. Zhu, W. Su, L. Lu, B. Li, X. Wang, and J. Dai, "Deformable detr: Deformable transformers for end-to-end object detection," in *International Conference on Learning Representations*, 2020.
- [91] S. Kim, S. Park, B. Na, and S. Yoon, "Spiking-yolo: Spiking neural network for energy-efficient object detection," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, pp. 11 270–11 277, 2020.
- [92] Y. Li, X. He, Y. Dong, Q. Kong, and Y. Zeng, "Spike calibration: Fast and accurate conversion of spiking neural network for object detection and segmentation," *arXiv preprint arXiv:2207.02702*, 2022.
- [93] H. Zhang, Y. Li, B. He, X. Fan, Y. Wang, and Y. Zhang, "Direct training high-performance spiking neural networks for object recognition and detection," *Frontiers in Neuroscience*, vol. 17, p. 1229951, 2023.
- [94] Q. Su, Y. Chou, Y. Hu, J. Li, S. Mei, Z. Zhang, and G. Li, "Deep directly-trained spiking neural networks for object detection," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 6555–6565.
- [95] W. Wang, J. Dai, Z. Chen, Z. Huang, Z. Li, X. Zhu, X. Hu, T. Lu, L. Lu, H. Li *et al.*, "Internimage: Exploring large-scale vision foundation models with deformable convolutions," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2023, pp. 14 408–14 419.
- [96] H. Zhang, C. Wu, Z. Zhang, Y. Zhu, H. Lin, Z. Zhang, Y. Sun, T. He, J. Mueller, R. Manmatha *et al.*, "Resnest: Split-attention networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 2736–2746.
- [97] C. Feichtenhofer, H. Fan, J. Malik, and K. He, "Slowfast networks for video recognition," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2019, pp. 6202–6211.
- [98] Z. Wang, Q. She, and A. Smolic, "Action-net: Multipath excitation for action recognition," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2021, pp. 13 214–13 223.
- [99] G. Bertasius, H. Wang, and L. Torresani, "Is space-time attention all you need for video understanding?" in *International Conference on Machine Learning*. PMLR, 2021, p. 4.
- [100] M. Yao, J. Hu, G. Zhao, Y. Wang, Z. Zhang, B. Xu, and G. Li, "Inherent redundancy in spiking neural networks," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023, pp. 16 924–16 934.
- [101] O. Roy and M. Vetterli, "The effective rank: A measure of effective dimensionality," in *2007 15th European Signal Processing conference*. IEEE, 2007, pp. 606–610.



Man Yao received the Ph.D. from Xi'an Jiaotong University, Xi'an, China, in 2023. He is currently an Assistant Professor at the Institute of Automation, Chinese Academy of Science (CASIA). He has published several papers in journals, including *Nature Communications*, *IEEE T-PAMI*, and *Neural Networks*, as well as at AI conferences such as *ICLR*, *NeurIPS*, *ICML*, and *ICCV*, where he served as the first author. His research interests include neuromorphic computing and dynamic neural networks.



Xuenui Qiu received the BS degree from University of Electronic Science and Technology of China, Chengdu, China, in 2024, where he is currently pursuing the Ph.D degree at the Institute of Automation, Chinese Academy of Sciences (CASIA). He has published papers at top artificial intelligence conferences including ICML, ICLR, AAAI, and served as a reviewer for top conferences. His research interests lie in AI4S, Neuromorphic computing, and Model Compression.



Tianxiang Hu received the BS degree from Xi'an Jiaotong University, Xi'an, China, in 2022, where he is currently working toward the ME degree in Institute of Automation, Chinese Academy of Sciences (CASIA). His current research interests include neuromorphic computing and spiking neural networks.



Jiakui Hu is currently pursuing the Ph.D. degree at the Institute of Medical Technology, Peking University, Beijing, China. He has published papers on top machine learning conferences including ICML, NeurIPS, and served as reviewers for top conferences. His research interests include brain-inspired algorithms and image restoration.



Yuhong Chou is currently pursuing the Ph.D degree at the Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China. He received the BS degree from Xi'an Jiaotong University, Xi'an, China, in 2024. His research interests lie in brain-like computing and foundation models.



Keyu Tian is currently enrolled in a Master's degree at Peking University supervised by Liwei Wang. He has published papers at top machine learning conferences including NeurIPS, ICLR, and served as reviewer for top conferences and journals. His research interests lie in self-supervised learning and generative modeling.



Jianxing Liao received the Bachelor and the Master degree from Xidian University, Xi'an, China, in 1997 and 2000, respectively. He is currently an Expert at the Advanced Computing and Storage Lab of Central Research Institute, Huawei Technologies Co., Ltd and an Industry Supervisor of the joint-master program between Huawei and the Southern University of Science and Technology, Shenzhen, China. His research focuses on brain-inspired computing and neuromorphic chips.



Luziwei Leng (IEEE Member) received the B.Sc. degree from the University of Electronic Science and Technology of China, Chengdu, China, in 2011, and the Master and the Doctoral degree from the Department of Physics, Heidelberg University, Germany in 2014 and 2019, respectively. He is currently a Principal Engineer at the Advanced Computing and Storage Lab of Central Research Institute, Huawei Technologies Co., Ltd and an Industry Supervisor of the joint-master program between Huawei and the Southern University of Science and Technology, Shenzhen, China. His research focuses on brain-inspired computing and machine learning, including spiking neural networks, event-based vision, bio-inspired learning rules and evolutionary algorithms, etc.



Bo Xu received the B.S. degree from Zhejiang University, Hangzhou, China, in 1988 and the M.S. and Ph.D. degrees from University of Chinese Academy of Sciences, Beijing, China, in 1992 and 1997, respectively. He currently is a Professor at Institute of Automation, Chinese Academy of Science (CASIA). And now is the director of CASIA, and associate director of CAS (Chinese Academy of Sciences) Center for Excellence in Brain Science and Intelligence Technology (CEBSIT). Prof. XU's research interests spans the area of speech recognition, spoken dialogue, multimodality information processing, brain-inspired learning, and decision intelligence.

He now serves as a steering committee member of the National Programme of New Generation Artificial Intelligence since 2017. He once was the chairman of Special Interesting Group in International Association of Chinese Spoken language Processing (SIG-ISCSLP) between 2001-2006., and the Coordinator of International Consortium for Speech Translation Advanced Research (C-Star). He was steering committee member of National High-Tech Program in the fields of intelligence information processing and interactive interfaces from 2004-2010. He won several prizes including the best paper of ISCSLP, outstanding young scientist of CAS and outstanding achievement in national media technology development. He has published more than 300 papers and holds 50 patents in the fields. He now serves as steering committee member of National Programme of New Generation Artificial Intelligence from 2017.



Guoqi Li (IEEE Member) received the Ph.D. degree from Nanyang Technological University, Singapore, in 2011. From 2011 to 2014, he was a Scientist with the Data Storage Institute and the Institute of High Performance Computing, Agency for Science, Technology and Research, Singapore. From 2014-2022, he was an Assistant professor and Associate professor at Tsinghua University, Beijing, China. Since 2022, he has been with the Institute of Automation, Chinese Academy of Sciences and the University of Chinese Academy of Sciences, where he is currently a Full professor. His current research interests include Brain-inspired Intelligence, Neuromorphic Computing and Spiking Neural Networks. He has authored or co-authored more than 170 papers in a number of prestigious journals including Nature, Nature Communications, Science Robotics, Proceedings of the IEEE, and top AI conference such as ICLR, NeurIPS, ICML, AAAI and so on.

Dr. Li has been actively involved in professional services such as serving as a Tutorial Chair, an International Technical Program Committee Member, a PC member, a Publication Chair, a Track Chair and workshop chair for several international conferences. He is an Editorial-Board Member for Control and Decision, and served as Associate Editors for Journal of Control and Decision and Frontiers in Neuroscience: Neuromorphic Engineering. He is a reviewer for Mathematical Reviews published by the American Mathematical Society and serves as a reviewer for a number of prestigious international journals and top AI conferences including ICLR, NeurIPS, ICML, AAAI and so on. He was the recipient of the 2018 First Class Prize in Science and Technology of the Chinese Institute of Command and Control, the Top ten scientific advances Award in China selected by the Ministry of science and technology, P.R. China as the backbone of the team member, and the 2020 Second Prize of Fujian Provincial Science and Technology Progress Award. He received the outstanding Young Talent Award of the Beijing Natural Science Foundation in 2021.